

Uncertainty-Aware Hand–Eye Calibration

Markus Ulrich  and Markus Hillemann 

Abstract—We provide a generic framework for the hand–eye calibration of vision-guided industrial robots. In contrast to traditional methods, we explicitly model the uncertainty of the robot in a stochastically founded way. Albeit the repeatability of modern industrial robots is high, their absolute accuracy typically is much lower. This uncertainty—especially if not considered—deteriorates the result of the hand–eye calibration. Our proposed framework does not only result in a high accuracy of the computed hand–eye pose but also provides reliable information about the uncertainty of the robot. It further provides corrected robot poses for a convenient and inexpensive robot calibration. Our framework is computationally efficient and generic in several regards. It supports the use of a calibration target as well as self-calibration without the need for known 3-D points. It optionally enables the simultaneous calibration of the interior camera parameters. The framework is also generic with regard to the robot type and, hence, supports antropomorphic as well as selective compliance assembly robot arm (SCARA) robots, for example. Simulated and real experiments show the validity of the proposed methods. An extensive evaluation of our framework on a public dataset shows a considerably higher accuracy than 15 state-of-the-art methods.

Index Terms—Automation, calibration, computer vision, hand–eye calibration, industrial robots, measurement uncertainty.

I. INTRODUCTION

HAND–EYE calibration is essential for applications that use vision-guided robots. It determines the 3-D rigid transformation (pose) between the robot and the camera (hand–eye pose). This enables measurements that are performed in the camera coordinate system to be transformed into the robot coordinate system. In bin-picking applications, for example, the pose of an object is determined in the coordinate system of the camera by using 3-D object recognition [1], [2], [3], [4]. To grasp the object, the object pose must be transformed into the coordinate system of the robot.

In general, there are two different scenarios of vision-guided robots [5, Ch. 3.12.2]. In the first scenario, the camera is mounted at the robot tool and is moved to different positions by the robot. In the second scenario, the camera is mounted stationary outside the robot without moving with respect to the robot base. The pose that must be determined by hand–eye calibration is the

pose of the camera with respect to the robot tool in the first scenario, or the pose of the camera with respect to the robot base in the second scenario. For simplicity, we focus on the first scenario with a moving camera in the following. However, our framework can be applied analogously to the second scenario with a stationary camera.

Traditional approaches for hand–eye calibration assume that the pose of the robot tool with respect to the robot base is known accurately. When discussing the accuracy of robots, it is necessary to distinguish between the pose repeatability (i.e., precision) and the pose accuracy of the robot [6]. Repeatability is a measure of the robot’s ability to move the tool repeatedly to the same pose. Accuracy is a measure of the robot’s ability to move its tool to a specified pose in 3-D space. Modern industrial robots typically offer a very high repeatability, which is in the range of 0.02–0.15 mm [7], [8], [9], [10]. For applications where the robot always moves to the exact same pose, which is trained in advance, a high repeatability is sufficient. However, for robots that are programmed offline and especially for vision-guided robots, obviously a high pose accuracy is important as well. Unfortunately, the pose accuracy of robots often is much lower than the pose repeatability. The position accuracy typically is in the range of 0.1–5.0 mm whereas the orientation accuracy ranges from about 0.2° up to several degrees [8], [9], [10].

It is possible to increase the accuracy with a robot calibration by up to an order of magnitude, in rare cases even up to the level of repeatability [7]. Unfortunately, robot calibration often requires high-precision measurement instruments like lasertrackers [8], and hence is expensive and time consuming. In this article, we combine all errors of the robot under the term uncertainty, which include errors that have an impact on repeatability and on accuracy.

Almost all traditional methods for hand–eye calibration do not explicitly model erroneous robot poses. Our proposed framework explicitly models the uncertainty of the robot in a stochastically founded way resulting in a higher accuracy of the computed hand–eye pose. Furthermore, it provides reliable information about the uncertainty of the robot, which otherwise would require high-precision measurement instruments. Since the framework also provides error-corrected robot poses, it can support a convenient and inexpensive robot calibration by assuming the error-corrected robot poses as ground-truth poses for the estimation of the robot parameters (i.e., Denavit–Hartenberg (DH) parameters [11], for example).

Most existing approaches for hand–eye calibration require the acquisition of multiple images of a calibration target. A few more flexible and user-friendly solutions avoid the use of calibration targets. Our framework supports both variants,

Manuscript received 1 August 2023; revised 20 October 2023; accepted 31 October 2023. Date of publication 6 November 2023; date of current version 15 December 2023. This paper was recommended for publication by Associate Editor J. Kelly and Editor P. Robuffo Giordano upon evaluation of the reviewers’ comments. (Corresponding author: Markus Ulrich.)

The authors are with the Institute of Photogrammetry and Remote Sensing, Karlsruhe Institute of Technology, 76131 Karlsruhe, Germany (e-mail: markus.ulrich@kit.edu; markus.hillemann@kit.edu).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TRO.2023.3330609>.

Digital Object Identifier 10.1109/TRO.2023.3330609

target-based calibration as well as self-calibration without the need for known 3-D points. Furthermore, for both variants, the framework offers the possibility to simultaneously calibrate the interior camera parameters if desired.

This publication is an evolved article. It extends our previously published work [12] in several aspects. We included and discussed additional relevant related work. We give details about the polynomial distortion model that supports a higher accuracy compared with the division model. We also included a comprehensive description of the structure of the parameter estimation including a visualization of the Jacobian, a flowchart, and a detailed pseudocode of the approach. All necessary partial derivatives have been analytically calculated and are now given in the Appendix B supporting a faster, more accurate, and hence, more robust computation. A new section contains details about the estimation of the variance components. We have also improved the implementation in a way that our framework can now be applied to calibration scenarios with a large number of points and/or calibration images. These optimizations, which are explained in depth in a new section, drastically reduce both memory consumption and computation time, making our framework more widely applicable. We also executed several new experiments in order to better assess the performance of our approach. Furthermore, we applied our implementation to an additional public dataset, which enables a comparison to a larger number of state-of-the-art methods and further illustrates the advantages of our uncertainty-aware hand-eye calibration. A new chapter contains in-depth discussions and possible future research questions. Finally, our method has become part of the HALCON¹ software from version 23.05 [13] on, which is freely available for academic use².

The rest of this article is organized as follows. We will give an overview of the related work in Section II. Section III introduces our generic framework for hand-eye calibration of uncertain robots. In Section IV, different aspects of our framework are extensively evaluated including a comparison to 15 state-of-the-art methods. Finally, Section V concludes this article.

II. RELATED WORK

Fig. 1 shows the coordinate systems that are relevant for the hand-eye calibration of a robot with a moving camera (for a stationary camera, see [5, Ch. 3.13], for example). Let ${}^{s_2}\mathbf{H}_{s_1}$ denote the 4×4 homogeneous transformation matrix that represents a rigid 3-D transformation of points from coordinate system s_1 to s_2 . One of the most common problem formulations of hand-eye calibration is based on closing the following chain of transformations [14]:

$${}^b\mathbf{H}_w = {}^b\mathbf{H}_t {}^t\mathbf{H}_c {}^c\mathbf{H}_w \quad (1)$$

with the coordinate systems world (w, WCS), camera (c, CCS), robot tool (t, TCS), and robot base (b, BCS), as well as the unknown poses ${}^t\mathbf{H}_c$ (hand-eye pose) and ${}^b\mathbf{H}_w$ (world-base pose). The unknown poses are typically determined by moving the

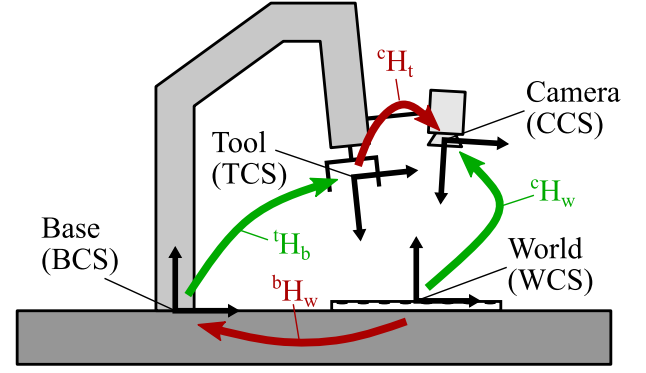


Fig. 1. Relevant 3-D coordinate systems, known or observed transformations (green), and unknown (red) transformations for hand-eye calibration, where a camera (eye) is rigidly mounted at the robot's end-effector (hand).

robot to different robot poses and taking an image of a calibration target at each pose. The calibration target defines the WCS. At each robot pose, ${}^b\mathbf{H}_t$ is queried from the robot controller and ${}^c\mathbf{H}_w$ is determined by Perspective-n-Point (PnP) algorithms or camera calibration like [15]. Equation (1) is often written as

$$\mathbf{Y} = \mathbf{A}_i \mathbf{X} \mathbf{B}_i \quad (2)$$

with the unknown poses $\mathbf{X}$ and $\mathbf{Y}$ and the observed poses $\mathbf{A}_i$ and $\mathbf{B}_i$ for each of the n robot poses ($i = 1, \dots, n$). Since the essential unknown is $\mathbf{X}$, we can eliminate $\mathbf{Y}$ by taking one pair of different robot poses i and j , yielding $\mathbf{A}_j^{-1} \mathbf{A}_i \mathbf{X} = \mathbf{X} \mathbf{B}_j \mathbf{B}_i^{-1}$. With $\mathbf{A} = \mathbf{A}_j^{-1} \mathbf{A}_i$ and $\mathbf{B} = \mathbf{B}_j \mathbf{B}_i^{-1}$ we obtain

$$\mathbf{A} \mathbf{X} = \mathbf{X} \mathbf{B} \quad (3)$$

where the movement of the tool is represented by $\mathbf{A}$ and the movement of the camera is represented by $\mathbf{B}$, when moving the robot from pose i to j .

There are several linear least-squares approaches that basically solve (2) or (3), e.g., [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], and [26]. They either determine the rotation and translation parts of the hand-eye pose successively or simultaneously. The latter has the advantage that rotation errors do not propagate and increase translation errors. Approaches that solve (3) require the selection of suitable pairs of robot poses to compute $\mathbf{A}$ and $\mathbf{B}$. Criteria for the suitability are proposed in [16], [23], and [27], for example. However, it is still difficult to ensure that the observed information ($\mathbf{A}_i$ and $\mathbf{B}_i$) is optimally exploited. Furthermore, these approaches do not explicitly model errors in the robot poses [16].

Because linear approaches typically minimize an algebraic error, their accuracy is limited. Therefore, they are often used to initialize a subsequent nonlinear optimization to obtain a higher accuracy. Most nonlinear approaches ([5, Ch. 3.13.5], [19], [20], [26], [28], [29], [30], [31], [32], [33]), which either minimize an algebraic or geometric error, also do not explicitly model errors in the robot poses. Some of them face the additional problem of weighting the error components of rotation and translation appropriately with respect to each other. Handling uncertainty in hand-eye calibration is a largely neglected topic [34]. In [14], for example, a weighted sum of the rotation and translation error

¹[Online]. Available: www.mvtec.com/products/halcon

²[Online]. Available: www.mvtec.com/company/mvtec-on-campus

parts is minimized where the weights for the error components are statistically derived. While this takes errors in the robot poses ${}^b\mathbf{H}_t$ into account, error-free camera poses ${}^c\mathbf{H}_w$ are assumed as input. In [35], rotation and translation parts of (3) are solved successively, whereas errors in $\mathbf{A}$ and $\mathbf{B}$ are propagated to $\mathbf{X}$. In [36], uncertainties of the camera and the robot poses are also considered with covariance matrices and a Cramer–Rao lower bound of the uncertainty of $\mathbf{X}$ is determined. In [37] and [38], the rotation and translation parts of $\mathbf{X}$ are solved simultaneously, the uncertainty of $\mathbf{X}$ is derived analytically, and the sensitivity to robot and camera pose noise is investigated. However, in [36], [37], and [38], an algebraic error is minimized. Furthermore, no information about the uncertainty of the robot is provided. Some nonlinear approaches do not require initial estimates and provide globally optimal solutions with respect to a certain cost function. In [39], this is achieved by solving a multivariate polynomial optimization problem over semi-algebraic sets using the method of convex linear matrix inequality relaxations. More recently, in [40], this is achieved by using a convex semidefinite programming relaxation. In the presence of noise, however, globally optimal solutions do not imply zero error with respect to the ground truth, but still differ from it [40].

Another class of approaches minimizes the reprojection error of 3-D world points on a calibration target (e.g., [41] and [42]) similar to camera calibration approaches like [15]. The appealing advantages are that it is unnecessary to explicitly estimate the camera pose in each image in preprocessing, no preselection of pose pairs is required, and a meaningful geometric error is minimized in the space of the observed erroneous measurements, i.e., the image points. Minimizing the reprojection error also enables the simultaneous estimation of the interior camera parameters. However, also these methods do not model errors in the robot poses. Recently, in [43], a pose graph optimization framework for hand–eye calibration was introduced, in which the errors of the robot poses are minimized in addition to the reprojection error of the points on a calibration target. Experiments showed the benefit of taking the uncertainty of robot poses into account. Unfortunately, no details are given about the stochastic model, the optimization, and how the different error components are weighted with respect to each other.

Self-calibration techniques are used to perform the hand–eye calibration without a calibration target. For the purpose of this article, self-calibration does not mean that the interior orientation of the camera is determined, but that no calibration target is used for the hand–eye calibration. In [30], unknown 3-D points are tracked in the image sequence obtained from a predefined robot motion. The interior camera parameters and the 3-D points are simultaneously estimated together with the hand–eye pose. A structure-from-motion (SfM) approach is used in [22], where the unknown scale factor of the SfM result is integrated into the equations. The idea is picked up in [31], where the post-processing step of enforcing the orthogonality of the rotation matrix is avoided by introducing the unknown scale factor into the equations of [19] and [21]. Because these self-calibration techniques minimize algebraic errors, the obtained accuracy is limited. Note that of course also self-calibration techniques can be applied to determine the movement of the camera to solve

(3), provided that a sufficient number of image features in a nondegenerate configuration is available.

Our proposed framework combines the mentioned benefits of minimizing a reprojection error, the advantage of stochastically modeling the uncertainty of all measured observations (image points and robot poses), the flexibility of either performing target-based calibration or self-calibration, and the possibility of either using known interior camera parameters or estimating them simultaneously.

Lastly, we want to highlight the differences of our framework to the approach described in [14], which at first glance could be considered similar to ours. One fundamental difference is that the approach in [14] assumes that the camera poses have been determined in advance with a calibration procedure and that these camera poses are error-free. In contrast, in our approach no camera poses need to be provided for input. The camera poses are determined only implicitly during the optimization. Therefore, by minimizing the reprojection error, uncertainties in the camera poses are considered implicitly in our approach as well. Furthermore, in [14] not the originally observed observations (image points) are modeled as erroneous quantities but, rather, functionally derived quantities from them, i.e., in form of one rotation and one translation error. The approach of [14] also does not allow to compute calibrated robot poses because errors in the camera poses and robot poses are mixed. Also, the weights that are automatically computed in [14] to balance translation and rotation errors might be biased by the assumption of error-free camera poses and, hence, cannot represent the robot accuracy. In contrast to [14], our approach models all originally observed and, thus, error-prone measurements directly as random variables (see Section III-F).

III. HAND-EYE CALIBRATION OF UNCERTAIN ROBOTS

Fig. 2 shows a flowchart of our uncertainty-aware hand–eye calibration framework. It gives a rough overview of the calibration flow and the variety of use cases. In particular, the flowchart highlights four important aspects of our framework.

- 1) The simultaneous calibration of the interior camera parameters is supported if no camera calibration is done beforehand.
- 2) Both target-based calibration as well as self-calibration are supported.
- 3) The automatic variance components estimation and its iterative nature.
- 4) The variety of outputs.

In this section, we will first describe the camera model and the calibration model, i.e., the relation between 3-D world points and their projection into the camera that is mounted on the end-effector of the robot. After a short introduction to parameter estimation models, we will introduce three alternative least-squares estimation models for hand–eye calibration and propose how to obtain initial values for the estimation. The section concludes with implementation details.

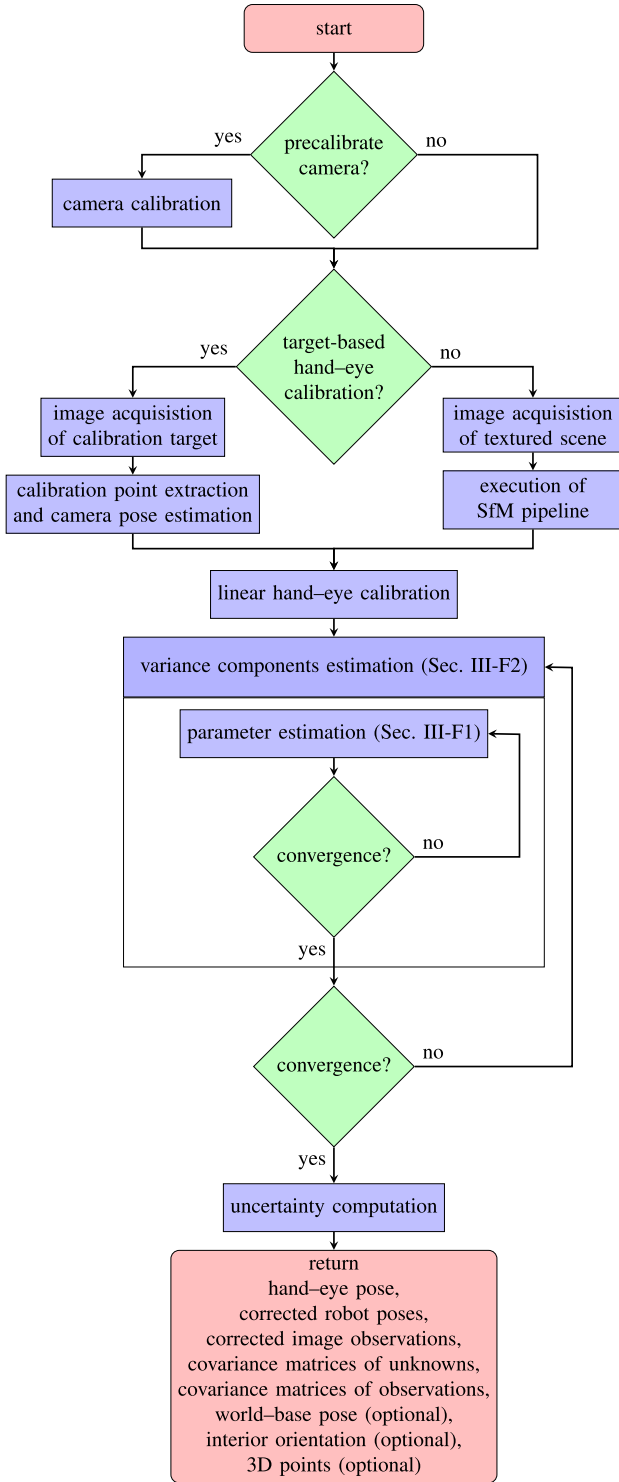


Fig. 2. Flowchart of our uncertainty-aware hand-eye calibration framework.

A. Camera Model

We represent the camera by the perspective camera model described in [5, Ch. 3.9.1]. A 3-D point $\mathbf{p}_w$ in the WCS is transformed to a point $\mathbf{p}_c$ in the CCS by

$$\mathbf{p}_c = \mathbf{R}\mathbf{p}_w + \mathbf{t} \quad (4)$$

where $\mathbf{t} = (t_x, t_y, t_z)^\top$ is a translation vector and $\mathbf{R}$ is a rotation matrix parameterized by Euler angles: $\mathbf{R} = \mathbf{R}_x(\alpha)\mathbf{R}_y(\beta)\mathbf{R}_z(\gamma)$ [see (25)–(27)]. The parameters $(t_x, t_y, t_z, \alpha, \beta, \gamma)$ describe the exterior orientation of the camera. With homogeneous coordinates (i.e., after extending $\mathbf{p}_w$ and $\mathbf{p}_c$ with a fourth coordinate of 1), the transformation can be written as a 4×4 homogeneous matrix

$$\mathbf{p}_c = {}^c\mathbf{H}_w\mathbf{p}_w = \begin{pmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix} \mathbf{p}_w. \quad (5)$$

The point $\mathbf{p}_c = (x_c, y_c, z_c)^\top$ is then projected into the image plane by

$$\begin{pmatrix} x_u \\ y_u \end{pmatrix} = \frac{c}{z_c} \begin{pmatrix} x_c \\ y_c \end{pmatrix} \quad (6)$$

with the principal distance c of the lens. Even if we will concentrate on a perspective lens, other camera models (e.g., for telecentric lenses) can be included in a straightforward way [5, Ch. 3.9]. Next, the undistorted point $(x_u, y_u)^\top$ is distorted to $(x_d, y_d)^\top$, e.g., by either the division model [44]

$$\begin{pmatrix} x_u \\ y_u \end{pmatrix} = \frac{1}{1 + \kappa r_d^2} \begin{pmatrix} x_d \\ y_d \end{pmatrix} \quad (7)$$

which models radial distortions by κ , or the polynomial model, which is based on [45]

$$\begin{pmatrix} x_u \\ y_u \end{pmatrix} = \begin{pmatrix} x_d(1 + K_1 r_d^2 + K_2 r_d^4 + K_3 r_d^6) + (P_1(r_d^2 + 2x_d^2) + 2P_2 x_d y_d) \\ y_d(1 + K_1 r_d^2 + K_2 r_d^4 + K_3 r_d^6) + (2P_1 x_d y_d + P_2(r_d^2 + 2y_d^2)) \end{pmatrix} \quad (8)$$

which models radial distortions by K_1, K_2 , and K_3 as well as decentering distortions by P_1 and P_2 . Here, $r_d^2 = x_d^2 + y_d^2$. While the division model can be inverted analytically, the polynomial model must be inverted by a numerical root finding algorithm [5, Ch. 3.9.1.3]. This is significantly slower than the analytical inversion in the division model. This is why we implemented the polynomial model the other way round compared with [45]. In machine vision, it is important that in online application the rectification of distorted measurements is fast, for example, when transforming image coordinates to measurements in the world. Thus, it is advantageous to compute the rectification instead of the distortion analytically [5].

Finally, the distorted point $(x_d, y_d)^\top$ is transformed into the image coordinate system

$$\mathbf{p}_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} = \begin{pmatrix} x_d/s_x + c_x \\ y_d/s_y + c_y \end{pmatrix} \quad (9)$$

where $(c_x, c_y)^\top$ is the principal point and s_x and s_y denote the pixel pitches on the sensor (x refers to the horizontal and y to the vertical axis of the image).

The six parameters $\mathbf{i} = (c, \kappa, s_x, s_y, c_x, c_y)^\top$ for the division model or the ten parameters $\mathbf{i} = (c, K_1, K_2, K_3, P_1, P_2, s_x, s_y, c_x, c_y)^\top$ for the polynomial model describe the interior orientation $\mathbf{i}$ of the camera.

B. Calibration Model

For hand-eye calibration, the tool of the robot is moved to n_r different poses and an image is acquired at each pose. For target-based calibration, the calibration target is placed at a fixed position within the workspace of the robot (see Fig. 1). In the self-calibration case, images are acquired from any sufficiently textured object or background scene instead. Let ${}^t\mathbf{H}_{b,j}$ be the robot pose that is queried from the robot controller at pose j ($j = 1, \dots, n_r$). Further, let the 3-D world points (given in the WCS) of the calibration target or in the scene be $\mathbf{p}_k$ ($k = 1, \dots, n_w$) and their 2-D projections into the image at robot pose j be $\mathbf{p}_{j,k}$. Then, we can describe the projection of a 3-D point into an image by

$$\mathbf{p}_{j,k} = \pi({}^c\mathbf{H}_t {}^t\mathbf{H}_{b,j} {}^b\mathbf{H}_w \mathbf{p}_k, \mathbf{i}) \quad (10)$$

where $\pi(\mathbf{p}_c, \mathbf{i})$ is the projection of the point $\mathbf{p}_c$, which is given in the CCS, into the image by successive execution of (6), the inverse of either (7) or (8), and (9), by using the parameters of the interior orientation $\mathbf{i}$. We denote the vector that contains the six transformation parameters of the robot poses ${}^t\mathbf{H}_{b,j}$ by $\mathbf{e}_{t,j}$, that of the unknown hand-eye pose ${}^c\mathbf{H}_t$ by $\mathbf{e}_c$, and that of the unknown pose ${}^b\mathbf{H}_w$ by $\mathbf{e}_b$ (in analogy with ${}^c\mathbf{H}_w$ above).

C. Introduction to Estimation Models

Estimation theory is used to estimate unknown parameters (unknowns) from given erroneous and redundant observed parameters (observations). Parameter estimation has a long tradition in photogrammetric computer vision and is often based on weighted least-squares estimation as a special case of Bayesian estimation [46].

Two of the most common models that are applied for this purpose are the Gauss–Markov model and the Gauss–Helmert model, both of which exist in a multitude of variations. One key advantage of these models is that the stochastic properties of the observed and unknown parameters are taken into account by the second moments of their distributions, i.e., their variances and covariances. Furthermore, a variety of established tools exist for evaluating the estimation results, making both models easy to apply in practice. The Gauss–Markov model assumes that the observations can be expressed as an explicit function of the unknowns. Since this assumption is not fulfilled in all practical cases, the more general but also less efficient Gauss–Helmert model is able to represent constraints between observations and unknowns via implicit functions [46].

D. Estimation in the Gauss–Markov Model

We distinguish between the functional model and the stochastic model [46]. The functional model describes the relations between the observations and the unknowns. In the stochastic model, the observations and the unknowns are treated as random variables with uncertainties, where the uncertainties are described by (co-)variances.

For error-free robot poses, the hand-eye calibration task can be formulated in the Gauss–Markov model [46, Ch. 4.4] because all uncertain observations $\mathbf{l}$ can be expressed as a function

$\mathbf{l} = \mathbf{f}(\mathbf{x})$ of the unknown parameters $\mathbf{x}$, where $\mathbf{f}$ describes the functional model and corresponds to (10).

The vector $\mathbf{l}$ contains the measured image points $\mathbf{l} = (\mathbf{p}_{1,1}^\top, \dots, \mathbf{p}_{1,n_w}^\top, \dots, \mathbf{p}_{n_r,n_w}^\top)^\top$. If a 3-D point is not visible in a certain image, the respective entry in $\mathbf{l}$ is simply omitted. Let n_i denote the total number of measured image points over all images. Then, the number of observations n_l is $2n_i$. For each observed image point, we set up two equations of the form of (10) yielding $2n_i$ equations. The vector $\mathbf{x}$ depends on the application scenario and is formed according to the following recipe.

- 1) For all scenarios, $\mathbf{x}$ contains the hand-eye pose: $\mathbf{x} := \mathbf{e}_c$. Note, however, that for selective compliance assembly robot arm (SCARA) robots, the z component t_z of the translation part of the hand-eye pose cannot be determined as all robot poses are parallel [47]. To be more specific, the optimization problem would become singular and yield a 1-D manifold of equivalent solutions. Therefore, for SCARA robots, t_z is (arbitrarily) set to 0, excluded from the optimization, and determined in postprocessing like proposed in [26], i.e., in this case, $\mathbf{x}$ contains only the five remaining parameters of $\mathbf{e}_c$.
- 2) For the scenario of a target-based calibration, $\mathbf{x}$ must be extended by the unknown $\mathbf{e}_b$: $\mathbf{x} := (\mathbf{x}^\top, \mathbf{e}_b^\top)^\top$. Note that for the self-calibration scenario, we can reconstruct the 3-D points directly in the BCS. Hence, we exclude $\mathbf{e}_b$ from the unknowns and internally set $\mathbf{e}_b = (0, 0, 0, 0, 0, 0)^\top$.
- 3) In the scenario, where the interior orientation of the camera is unknown and shall be simultaneously estimated, we further extend $\mathbf{x} := (\mathbf{x}^\top, \mathbf{i}^\top)^\top$. Note that s_y is typically excluded from the calibration [5, Ch. 3.9.4.2].
- 4) For the self-calibration scenario, $\mathbf{x}$ must be further extended by the coordinates of the 3-D points: $\mathbf{x} := (\mathbf{x}^\top, \mathbf{p}_1^\top, \dots, \mathbf{p}_{n_w}^\top)^\top$.

Let the final number of unknown variables in $\mathbf{x}$ be n_x . It should be noted that the singularity in the Euler angle parameterization is not critical when perturbations with respect to the initial poses of $\mathbf{e}_c$ and $\mathbf{e}_b$ are used. These perturbations will be close to the identity transformation, such that the singularity of the Euler angle parameterization is irrelevant. Regardless, our framework supports the use of alternative pose parameterizations if desired. The initial poses are obtained by a noniterative approach, e.g., by the dual-quaternion-based approach in [21, Sec. III-G].

The stochastic model specifies the statistical properties of the observation process. Assuming that the image points are uncorrelated and are measured with the same accuracy, we set the $n_l \times n_l$ weight coefficient matrix of the observations to the identity: $\mathbf{Q}_l = \mathbf{I}$. Note that it is sufficient to only specify the ratio of the variances of the observations in $\mathbf{Q}_l$. Their absolute values will be estimated later by scaling $\mathbf{Q}_l$ with the variance factor [see (13) and (14)].

For linearization, we compute the $2n_i \times n_x$ Jacobian $\mathbf{A}^3$ of $\mathbf{f}$ at the initial values of the unknowns $\mathbf{x}^{(0)}$ (cf. Section III-G)

$$\mathbf{A} = \left. \frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}^{(0)}} \quad (11)$$

³Note that from here on, $\mathbf{A}$ and $\mathbf{B}$ represent Jacobians in contrast to Section II.

With $\Delta l = l - f(x^{(0)})$ and the weight matrix $\mathbf{P}_{ll} = \mathbf{Q}_{ll}^{-1}$, we can calculate the corrections for the unknowns by solving

$$\mathbf{A}^\top \mathbf{P}_{ll} \mathbf{A} \Delta \hat{x} = \mathbf{A}^\top \mathbf{P}_{ll} \Delta l \quad (12)$$

for $\Delta \hat{x}$ by sparse Cholesky decomposition, for example⁴. For details about the Gauss–Markov model, including the derivation of (12), the interested reader is referred to [46, Ch. 4.4.1].

Then, $\hat{x}^{(1)} = x^{(0)} + \Delta \hat{x}$, and (11) and (12) are repeatedly applied until convergence. This corresponds to the minimization of the reprojection error of the 3-D points. After convergence, the covariance matrix of the original observations is obtained by

$$\mathbf{C}_{ll} = \hat{\sigma}_0^2 \mathbf{Q}_{ll} \quad (13)$$

with the variance factor

$$\hat{\sigma}_0^2 = \hat{v}^\top \mathbf{P}_{ll} \hat{v} / r \quad (14)$$

the residuals $\hat{v} = \mathbf{A} \Delta \hat{x} - \Delta l$, and the redundancy $r = n_l - n_x$. The corrected observations are obtained by $\hat{l} = l + \hat{v}$. The covariance matrix of the estimated unknowns is obtained by variance propagation

$$\mathbf{C}_{\hat{x}\hat{x}} = \hat{\sigma}_0^2 (\mathbf{A}^\top \mathbf{P}_{ll} \mathbf{A})^{-1}. \quad (15)$$

Note that the special case of this approach with target-based calibration and $\mathbf{Q}_{ll} = \mathbf{I}$ corresponds to the rp_1 method (fixed interior orientation) or rp_2 method (simultaneously estimated interior orientation) in [42].

E. Estimation in the Gauss–Helmert Model

In order to consider uncertain robot poses, these must be introduced as erroneous observations in addition to the image coordinates. Hence, the observations can no longer be expressed explicitly as a function of the unknowns. For this purpose, parameter estimation must be performed in the Gauss–Helmert model [46, Ch. 4.8]. In the Gauss–Helmert model, our functional model becomes $f(x, l) = 0$.

In this model, the vector l contains the observed image points as well as the robot poses $l = (p_{1,1}^\top, \dots, p_{1,n_w}^\top, \dots, p_{n_r,n_w}^\top, e_{t,1}^\top, \dots, e_{t,n_r}^\top)^\top$. The number of observation equations is still $2n_l$. However, the number of observations now becomes $n_l = 2n_i + 6n_r$. The vector x of unknowns is identical to that in the Gauss–Markov model.

In comparison with the Gauss–Markov model, the stochastic model must also consider the uncertainties of the robot poses. Tests on real systems showed that the errors in the robot poses are zero-mean Gaussian distributed [14]. But even if assuming uncorrelated observations, we still would have to know the relative accuracy between the three observation groups (image coordinates, Euler angles of the robot poses, and translation components of the robot poses) to set up $\mathbf{Q}_{ll}$. The absolute accuracy values are not required as input. However, they can be computed from the estimation result. Therefore, we initialize $\mathbf{Q}_{ll}$ with reasonable values and use the concept of variance components [46] to estimate the true variances of the observations (see below). We could also initialize $\mathbf{Q}_{ll} = \mathbf{I}$, but then

the number of iterations to estimate the variance components would be unnecessarily high. For a higher efficiency, in our implementations we initially set the standard deviations of the image points to $\sigma_i = 0.1$ pixels of the Euler angles to $\sigma_a = 0.1^\circ$, and of the translation components to $\sigma_t = 1$ mm. Then

$$\mathbf{Q}_{ll} = \text{diag}(\text{rep}(\sigma_i^2, 2n_i), \text{rep}([\sigma_a^2, \sigma_a^2, \sigma_a^2, \sigma_t^2, \sigma_t^2, \sigma_t^2], n_r)) \quad (16)$$

where the function $\text{rep}(y, n)$ generates a vector containing n copies of y .

In the Gauss–Helmert model, the Jacobian $\mathbf{A}$ is the same as in the Gauss–Markov model. However, in the Gauss–Helmert model, we have to linearize f additionally with respect to the observations yielding the $2n_l \times n_l$ Jacobian

$$\mathbf{B} = \left. \frac{\partial f(x, l)}{\partial l} \right|_{x=x^{(0)}}. \quad (17)$$

Note that the upper left $2n_i \times 2n_i$ submatrix of $\mathbf{B}$ is a diagonal matrix with all diagonal elements set to -1 . With $w = f(x^{(0)}, l)$, we can calculate the corrections for the unknowns by solving

$$\mathbf{A}^\top (\mathbf{B} \mathbf{Q}_{ll} \mathbf{B}^\top)^{-1} \mathbf{A} \Delta \hat{x} = -\mathbf{A}^\top (\mathbf{B} \mathbf{Q}_{ll} \mathbf{B}^\top)^{-1} w \quad (18)$$

for $\Delta \hat{x}$. For details about the Gauss–Helmert model including the derivation of (18), the interested reader is referred to [46, Ch. 4.8].

After convergence, we estimate a variance component for each of the three observation groups. This yields corrected values for σ_i , σ_a , and σ_t , and, thus, a corrected $\mathbf{Q}_{ll}$ (for details about variance components estimation see Section III-F2). Then, the whole estimation process (which is an iterative process by itself) is repeated until the variance components converge. The covariance matrix of the observations is obtained by $\mathbf{C}_{ll} = \hat{\sigma}_0^2 \mathbf{Q}_{ll}$ with the redundancy $r = 2n_l - n_x$. The covariance matrix of the estimated unknowns is obtained by $\mathbf{C}_{\hat{x}\hat{x}} = \hat{\sigma}_0^2 (\mathbf{A}^\top (\mathbf{B} \mathbf{Q}_{ll} \mathbf{B}^\top)^{-1} \mathbf{A})^{-1}$.

In essence, this model allows us to simultaneously minimize the reprojection error and the error in the robot poses by automatically weighting the error terms with respect to each other based on the computed statistics.

F. Estimation in the Gauss–Markov Model With Fictitious Observations

1) *Parameter Estimation:* Because of the expensive matrix computations in the Gauss–Helmert model, we also implemented a more efficient variant of the Gauss–Markov model that is equivalent to the Gauss–Helmert model ([48], [49]) by using the concept of fictitious observations for the robot poses.

The key idea is to introduce the uncertain robot poses simultaneously as observations and as unknowns. The first part of the functional model is still $l = f(x)$. In contrast to Section III-D, however, we add six additional observation equations of the form $e_{t,j} = f_2(e_{t,j})$ for each robot pose j , where f_2 is the identity in our case. This results in $n_l = 2n_i + 6n_r$ observation equations. Hence, the vector l contains the observed image points and the robot poses $l = (p_{1,1}^\top, \dots, p_{1,n_w}^\top, \dots, p_{n_r,n_w}^\top, e_{t,1}^\top, \dots, e_{t,n_r}^\top)^\top$ as in Section III-E. Simultaneously, we also introduce the robot

⁴In statistics, the hat operator $\hat{\cdot}$ denotes an estimated or fitted value.

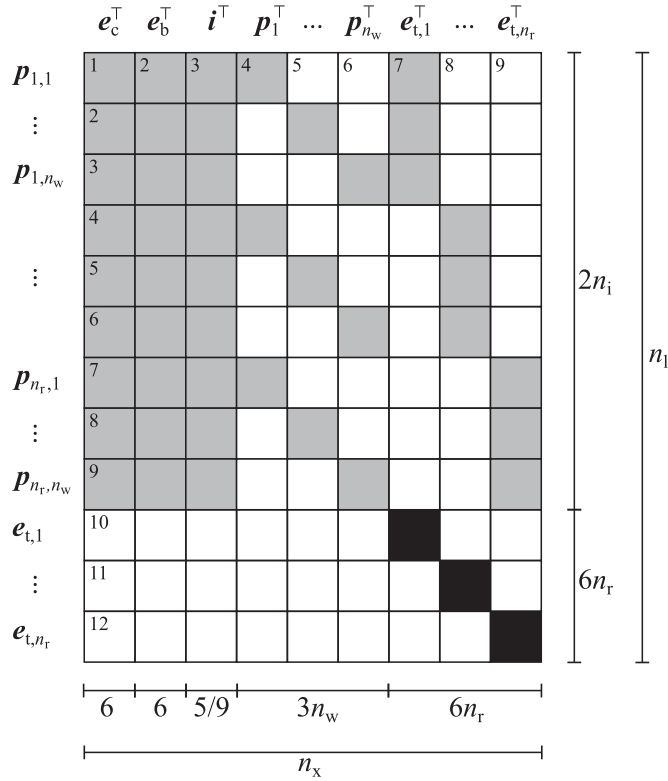


Fig. 3. Structure of the Jacobian $\mathbf{A}$ in the Gauss–Markov model with fictitious observations. The rows represent observations, the columns represent unknowns. White blocks indicate that all entries are 0, gray blocks indicate partial derivatives that are typically different from 0, and black blocks indicate the identity matrix. Block rows 1 to 9 represent the measured image points, block rows 10 to 12 represent the robot poses, i.e., the fictitious observations. Block column 1 represents the hand–eye pose. Block column 2 represents the pose of the calibration target and is omitted in the self-calibration case. Block column 3 represents the interior camera parameters, which, depending on the lens distortion model, contains either five or nine elements when excluding s_y from the calibration. Block column 3 is omitted if the camera parameters are already known and should not be estimated. Block columns 4 to 6 represent the 3-D world points, and hence are omitted in the case of target-based calibration. Block columns 7 to 9 represent the robot poses.

poses as unknowns by extending $\mathbf{x} := (\mathbf{x}^\top, \mathbf{e}_{t,1}^\top, \dots, \mathbf{e}_{t,n_r}^\top)^\top$, yielding n_x unknowns.

Because $\mathbf{l}$ is identical to that in the Gauss–Helmert model, we apply the same stochastic model and initialize $\mathbf{Q}_{ll}$ by using (16).

The computation of the $(2n_i + 6n_r) \times n_x$ Jacobian $\mathbf{A}$ and of the corrections $\Delta\hat{\mathbf{x}}$ is done by (11) and (12). Note that in this case the lower left part of $\mathbf{A}$ contains zeros and the lower right part the identity $\mathbf{I}_{6n_r \times 6n_r}$. Fig. 3 shows the complete structure of the Jacobian $\mathbf{A}$.

For the computation of $\mathbf{A}$, an analytic calculation of the partial derivatives is clearly preferable over a numerical calculation because in our case it is faster and the derivatives are more accurate, which leads to a faster convergence of the optimization. All partial derivatives are given in the Appendix B.

After convergence, the variance components are estimated as described in Section III-F2. Finally, $\mathbf{C}_{ll} = \hat{\sigma}_0^2 \mathbf{Q}_{ll}$, $\hat{\mathbf{l}}$, and $\mathbf{C}_{\hat{\mathbf{x}}\hat{\mathbf{x}}}$ are computed as described in Section III-D.

2) *Variance Components Estimation:* To estimate the variance components of the three observation groups we have to

generalize the stochastic model of the Gauss–Markov model [46, Ch. 4.2.4]. To put it simply, it is no longer sufficient to calculate a single global variance factor $\hat{\sigma}_0^2$ as in Section III-D. Instead, we need to estimate an individual variance factor for each observation group. This enables the correct modeling of the stochastic properties of each observation group. Note that because we are interested in the absolute values of the variances, two variance factors that model only the ratio between the three observation groups would not be sufficient. The steps that are necessary for variance components estimation are described in this section.

- 1) First, we compute the weight coefficient matrix of the estimated unknowns $\hat{\mathbf{x}}$

$$\mathbf{Q}_{\hat{\mathbf{x}}\hat{\mathbf{x}}} = (\mathbf{A}^\top \mathbf{P}_{ll} \mathbf{A})^{-1}. \quad (19)$$

Note that this computation step was already performed during parameter estimation (see (12) in Section III-D and III-F1).

- 2) Then, we apply error propagation to obtain the weight coefficient matrix of the corrected observations $\hat{\mathbf{l}}$

$$\mathbf{Q}_{\hat{\mathbf{l}}\hat{\mathbf{l}}} = \mathbf{A} \mathbf{Q}_{\hat{\mathbf{x}}\hat{\mathbf{x}}} \mathbf{A}^\top. \quad (20)$$

- 3) From this, we compute the weight coefficient matrix of the estimated residuals $\hat{\mathbf{v}}$

$$\mathbf{Q}_{\hat{\mathbf{v}}\hat{\mathbf{v}}} = \mathbf{Q}_{ll} - \mathbf{Q}_{\hat{\mathbf{l}}\hat{\mathbf{l}}} \quad (21)$$

and the so-called redundancy matrix

$$\mathbf{R} = \mathbf{Q}_{\hat{\mathbf{v}}\hat{\mathbf{v}}} \mathbf{P}_{ll} = \mathbf{I} - \mathbf{Q}_{\hat{\mathbf{l}}\hat{\mathbf{l}}} \mathbf{P}_{ll}. \quad (22)$$

The main diagonal of $\mathbf{R}$ contains the redundancy numbers, i.e., the contribution of a single observation to the total redundancy r , with $\sum_{i=1}^{n_l} R_i = r$ and R_i being the main diagonal element of $\mathbf{R}$ at row and column i . Recall that the total redundancy is the difference between the number of observations and the number of unknowns $r = n_l - n_x$.

- 4) Let I_i , I_a , and I_t be the sets of observation indices that contain the observed image point coordinates, the observed Euler angles of the robot poses, and the observed translation components of the robot poses, respectively. Further, let $\hat{\mathbf{v}}_i$, $\hat{\mathbf{v}}_a$, and $\hat{\mathbf{v}}_t$ be vectors that contain the subset of the elements of the residual vector $\hat{\mathbf{v}}$ that correspond to the respective observation group and let $\mathbf{P}_{ll,i}$, $\mathbf{P}_{ll,a}$, and $\mathbf{P}_{ll,t}$ be matrices that contain the subset of the rows and columns of the weight matrix $\mathbf{P}_{ll}$ that correspond to the respective observation group. Then, the variance components of the three observation groups can be computed as

$$\hat{\sigma}_{0,i}^2 = \frac{\hat{\mathbf{v}}_i^\top \mathbf{P}_{ll,i} \hat{\mathbf{v}}_i}{\sum_{i \in I_i} R_i}, \hat{\sigma}_{0,a}^2 = \frac{\hat{\mathbf{v}}_a^\top \mathbf{P}_{ll,a} \hat{\mathbf{v}}_a}{\sum_{i \in I_a} R_i}, \hat{\sigma}_{0,t}^2 = \frac{\hat{\mathbf{v}}_t^\top \mathbf{P}_{ll,t} \hat{\mathbf{v}}_t}{\sum_{i \in I_t} R_i}. \quad (23)$$

- 5) By multiplying the initially chosen variances of the observations with the variance components, we obtain improved estimates of the variances

$$\sigma_i^2 \leftarrow \hat{\sigma}_{0,i}^2 \sigma_i^2, \quad \sigma_a^2 \leftarrow \hat{\sigma}_{0,a}^2 \sigma_a^2, \quad \sigma_t^2 \leftarrow \hat{\sigma}_{0,t}^2 \sigma_t^2. \quad (24)$$

- 6) They are used to update the stochastic model. The update is performed by setting up a new weight coefficient matrix

$\mathbf{Q}_{ll}$ where the entries on its main diagonal are set to the new values of σ_i^2 , σ_a^2 , and σ_t^2 .

Then, the parameter estimation is restarted with the original observations but with the new matrix $\mathbf{Q}_{ll}$. The process is repeated until the variance components converge, i.e., $\hat{\sigma}_{0,i}^2$, $\hat{\sigma}_{0,a}^2$, and $\hat{\sigma}_{0,t}^2$ are all sufficiently close to 1.0. This is typically the case after three to five iterations.

Note that when estimating variance components, the global variance factor $\hat{\sigma}_0^2$ does not need to be computed by using (14) because convergence of the variance components automatically ensures that $\hat{\sigma}_0^2 = 1.0$. For more details about variance components estimation, the interested reader is referred to [46].

G. Initialization of the Unknowns

Because of the nonlinearity of the optimization problem, initial values for the unknowns need to be provided. We propose the following initialization for $\mathbf{x}$:

- 1) Initial values for the hand-eye pose $\mathbf{e}_c$ and for $\mathbf{e}_b$ are obtained from a linear approach (e.g., [21] for anthropomorphic robots or [26] for SCARA robots).
- 2) Initial values for the interior orientation are obtained from the data sheets of the camera (s_x and s_y) and the lens (c). The principal point $(c_x, c_y)^\top$ is set to the image center and the distortion coefficients are set to 0.
- 3) For self-calibration, a SfM pipeline (e.g., COLMAP [50]) is applied to the acquired images. It returns the camera parameters $\mathbf{i}$, the reconstructed 3-D points $\mathbf{p}_k$, and for each image j the extracted 2-D keypoints $\mathbf{p}_{j,k}$, and the exterior orientation ${}^c\mathbf{H}_{w,j}$.

In the self-calibration case, we fix the inherent unknown scale factor of SfM in the initial values for $\mathbf{p}_k$ and ${}^c\mathbf{H}_{w,j}$ by applying the following steps: For all pairs of robot poses $j1$ and $j2$, we compute the camera movement ${}^c\mathbf{H}_{c1} = {}^c\mathbf{H}_{w,j2} {}^c\mathbf{H}_{w,j1}^{-1}$ and the tool movement ${}^t\mathbf{H}_{t1} = {}^t\mathbf{H}_{b,j2} {}^t\mathbf{H}_{b,j1}^{-1}$. We, then, convert both rigid 3-D transformations to screw parameters [21]. From the screw congruency theorem [18], we know that the screw translation parameter d_t of the tool movement is identical to the screw translation parameter d_c of the camera movement. Therefore, the ratio d_t/d_c reflects the unknown scale factor of the SfM. For a higher robustness, the median over the ratios of all pairs is computed while ignoring ratios, for which d_t or d_c is below a noise threshold. Finally, the scale factor is used to correct the initial values for $\mathbf{p}_k$ and ${}^c\mathbf{H}_{w,j}$ before starting the optimization.

H. Implementation of the Gauss–Markov Model With Fictitious Observations

Pseudocode of the proposed uncertainty-aware hand-eye calibration with the Gauss–Markov model with fictitious observations (GMF) is given in Fig. 11 in the Appendix A.

In this section, details about an efficient implementation of the parameter estimation in the Gauss–Markov model with fictitious observations are given. Even though the computations in this model are less demanding than in the Gauss–Helmert model, a straightforward implementation of the presented algorithm

would still result in very long computation times and a huge memory consumption, especially for a large number of observed image points. Therefore, we will describe how to optimize the implementation while preserving the exact result.

- 1) A first set of optimizations is based on the fact that $\mathbf{Q}_{ll}$ is a diagonal matrix. Hence, $\mathbf{P}_{ll}$ is efficiently obtained by computing the reciprocals of the main diagonal elements of $\mathbf{Q}_{ll}$. Furthermore, when computing the corrections of the unknowns by solving (12), two multiplications can be sped up. First, the matrix multiplication of $\mathbf{P}_{ll}$ with $\mathbf{A}$ is replaced by scaling each row of $\mathbf{A}$ by the corresponding diagonal element of $\mathbf{P}_{ll}$. Second, the matrix multiplication of $(\mathbf{A}^\top \mathbf{P}_{ll} \mathbf{A})^{-1} \mathbf{A}^\top$ with $\mathbf{P}_{ll}$ is replaced by scaling each column of the former matrix by the corresponding diagonal element of $\mathbf{P}_{ll}$. Each of both replacements reduces the number of calculation operations from $2n_1^2 n_x$ to $n_1 n_x$.
- 2) A second set of optimizations can be applied because in Section III-F2 we saw that the entire redundancy matrix $\mathbf{R}$ does not need to be computed. Instead, it is sufficient to know its main diagonal to compute the variance components. Therefore, and because $\mathbf{P}_{ll}$ is a diagonal matrix as well, it follows from (22) that also for $\mathbf{Q}_{\hat{\mathbf{v}}\hat{\mathbf{v}}}$ only its main diagonal needs to be computed. Furthermore, we can conclude from (21) that the same holds for $\mathbf{Q}_{\hat{\mathbf{u}}\hat{\mathbf{u}}}$, too. Note that the right product in (20), i.e., $\mathbf{Q}_{\hat{\mathbf{x}}\hat{\mathbf{x}}} \mathbf{A}^\top$, is already available from the computation of the corrections of the unknowns in (12). Hence, the elements of the main diagonal of $\mathbf{Q}_{\hat{\mathbf{u}}\hat{\mathbf{u}}}$ can be efficiently computed by the scalar products of each row of $\mathbf{A}$ with the corresponding column of $\mathbf{Q}_{\hat{\mathbf{x}}\hat{\mathbf{x}}} \mathbf{A}^\top$. According to (22), each resulting diagonal element is multiplied by the corresponding diagonal element of $\mathbf{P}_{ll}$ and subtracted from 1.0 to obtain the corresponding element of $\mathbf{R}$. This drastically reduces the number of calculation operations from $2n_1^4 + (2n_x + 1)n_1^2 + 2n_x^2 n_1$ to $2n_x n_1$.
- 3) Also the computation of the numerators in (23) can be sped up by computing the elementwise product of the main diagonal of $\mathbf{P}_{ll}$ and the squared elements of $\hat{\mathbf{v}}$ and summing up the results. This reduces the number of calculation operations from $2n_1^2 + 2n_1$ to $3n_1$ for each observation group⁵.
- 4) After the convergence of the variance components, the computation of the statistics can be optimized as well: The computation of $\hat{\sigma}_0^2$ is optimized in analogy with the computation of the variance components described above. The main diagonal of the covariance matrix of the original observations $\mathbf{C}_{ll}$ is obtained by scaling the main diagonal of $\mathbf{Q}_{ll}$ with $\hat{\sigma}_0^2$.

Through the described optimizations, we not only speed up the calculations but also manage to avoid the computation of matrices of size $n_1 \times n_1$. This is important because n_1 may become huge in practice. To give one example, the optimizations reduce the overall memory consumption of the hand-eye calibration from approximately 3.9 GB to 100 MB and the computation

⁵Note that here, n_1 exceptionally represents the number of observations in the respective observation group.

time from 72 to 2 s (for $n_r = 25$ robot poses and $n_w = 200$ world points, which results in $n_l = 10150$).

IV. EVALUATION

This section presents the results of multiple experiments that show the validity and the advantages of our uncertainty-aware hand-eye calibration. We will first evaluate different selected aspects of our proposed model based on simulated data that we generated. Afterward, experiments on real data are intended to demonstrate the advantages in application scenarios. These include experiments with our own robot as well as evaluations on a public dataset of a real laboratory. Finally, a comprehensive comparison with 15 state-of-the-art methods based on a public dataset of synthetic images completes the section.

A. Model Validation on Simulated Data

We simulated an uncertain robot by adding zero-mean Gaussian noise to its robot poses ${}^t\mathbf{H}_{b,j}$ with a standard deviation of 1 mm in the translation components and of 0.1° in the orientation components. To the image points, we applied noise with a standard deviation of 0.1 pixels.

For calibration, we simulated a planar calibration target with $n_w = 40$ points and a diameter of about 1 m and placed it in the workspace of the robot. We set the hand-eye pose ${}^c\mathbf{H}_t$ to different values with a translation of about 10 cm and different orientations. For the camera of resolution 1280×1024 we assumed the division model and set $c = 8$ mm, $\kappa = 2000 \text{ m}^{-2}$, $s_x = 5.21 \text{ } \mu\text{m}$, $s_y = 5.2 \text{ } \mu\text{m}$, and $(c_x, c_y) = (645, 502)$ pixels. We generated 40 random robot poses ${}^t\mathbf{H}_{b,j}$ in its 1 m^3 workspace, such that at least 90% of the calibration points were visible in the image.

First, we calibrated the interior orientation of the camera by using the acquired images and the approach of [5, Ch. 3.9.4]. Then, for hand-eye calibration we applied the linear approach of Daniilidis [21], the nonlinear approach of Steger [5, Ch. 3.13.5], the approach based on the Gauss–Markov model of Tabb [42, Sec. III-D, rp1], which is equivalent to the rp_1 method, and the Gauss–Markov model with fictitious observations of Section III-F (GMF). All nonlinear approaches were initialized with the result in [21]. In all experiments, the results of the Gauss–Helmert model of Section III-E were identical to that of GMF (up to numerical inaccuracies). This shows the validity of using the more efficient GMF instead of the Gauss–Helmert model and also makes the presentation of the results of the Gauss–Helmert model superfluous. We repeated each experiment 30 times and display the mean values in the following.

Fig. 4 shows the translation and rotation error of the computed hand-eye poses ${}^c\mathbf{H}_t$. Let $\mathbf{H}$ represent the computed pose and $\mathbf{H}_{gt}$ the ground truth pose. We compute the translation error as the length of the translation part of $\mathbf{H} - \mathbf{H}_{gt}$. For the rotation error, we compute the magnitude of the Rodrigues rotation of $\mathbf{H}\mathbf{H}_{gt}^{-1}$. As expected, the errors of all approaches generally decrease when increasing the number of robot poses. GMF performs best in all cases. Furthermore, the advantage of modeling the uncertainty of the robot can be clearly seen from the difference between Tabb rp_1 , which is equivalent to the Gauss–Markov model of

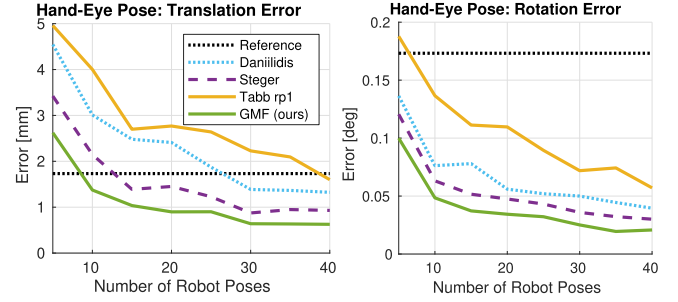


Fig. 4. Translation and rotation error of the hand-eye pose for different number of robot poses n_r and different approaches: Daniilidis [21], Steger [5], Tabb [42, rp1], GMF (ours). “Reference” visualizes the uncertainty of the simulated robot, i.e., the L2-Norm of the standard deviations applied to the robot poses.

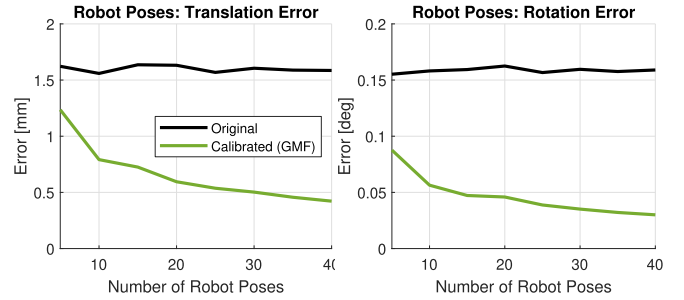


Fig. 5. Translation and rotation error of the robot poses ${}^t\mathbf{H}_{b,j}$ before (black) and after (green) GMF calibration.

Section III-D, and GMF. Note that the translation error might even exceed the uncertainty of the robot especially for fewer robot poses.

Fig. 5 shows the translation and rotation error of the robot poses before and after GMF calibration (the calibrated robot poses are contained in $\hat{\mathbf{l}}$). GMF is able to decrease the errors in the robot poses to about 20%–25%. Thus, GMF not only returns an accurate hand-eye pose but also enables a convenient vision-based robot calibration where the calibrated robot poses are assumed as ground-truth poses for the estimation (identification) of the robot parameters (e.g., DH parameters [11]).

To analyze the benefit of estimating the variance components, we fixed the number of robot poses to 40 and set the standard deviation of the rotation components of the simulated robot poses to 0.3° . Then, we varied the standard deviation of the translation components of the robot from 0 to 7.5 mm and repeated each experiment 30 times. The mean error of the hand-eye pose is shown in Fig. 6. As before, we initialized $\mathbf{Q}_l$ with $\sigma_a = 0.1^\circ$ and $\sigma_t = 1$ mm. The positive effect of estimating the variance components on the accuracy is obvious and increases the more the assumed ratio deviates from the true ratio of standard deviations.

Another advantage of estimating the variance components is that a meaningful accuracy of the robot is returned in $\mathbf{C}_l$, which otherwise would require costly robot calibration. On average, the accuracy (i.e., the standard deviations) of the robot was computed with an error of 61.1% in translation and 53.3% in rotation without estimating variance components, and with an error of

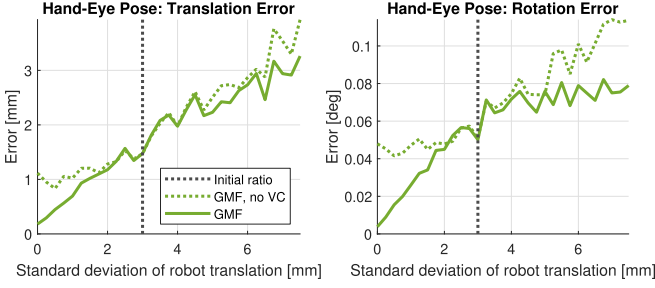


Fig. 6. Translation and rotation error of the hand-eye pose with (GMF) and without (GMF, no VC) estimating variance components. The standard deviation of the rotation components of the simulated robot was set to 0.3° and the standard deviation of the translation components was varied. The translation standard deviation that corresponds to the initial ratio $0.3 \cdot \sigma_t / \sigma_a$ mm is indicated by the vertical dotted line.

only 0.8% and 1.0% when estimating variance components. The mean standard deviation of the image points was estimated as 0.14 pixels without, and correctly as 0.10 pixels with estimating variance components.

In general, the experiments indicate that the hand-eye calibration is solvable in the proposed way. They show that the chosen problem formulation is not ill-posed and the system of normal equations (12) does not become singular. Furthermore, they show that the topography of the error surface allows the optimization to converge to the desired minimum. The experiment also indicates that the initial values that are obtained by the linear approach of [21] are sufficiently accurate.

Next, we simulated a robot with systematic errors. For this, we took the DH parameters [11], [51] of a UR3e robot arm and added systematic errors to the parameters of each of its six axes: We applied random systematic offsets with a standard deviation of 0.05 mm to the link length parameters a and to the link offset parameters d , random systematic offsets with a standard deviation of 0.02° to the link twist parameters α and to the joint angle parameters θ , and a random systematic scaling $1 + \epsilon$ of the joint angle parameters θ with a standard deviation of 0.0001 for ϵ . Then, we simulated 40 robot poses by applying the forward kinematics [51] once with the erroneous DH parameters and once with the original DH parameters. We then projected the 3-D points of a calibration target with 837 points for each robot pose into the camera image by using the erroneous robot pose and added noise to the image points with a standard deviation of 0.1 pixels. The robot poses that were computed with the original DH parameters were used as input to the same hand-eye calibration approaches as before. Again, all nonlinear approaches were initialized with the result in [21]. For now, the camera parameters were assumed to be known. For the resulting hand-eye poses, we again computed the error in the translation part and in the rotation part. We repeated the experiment 20 times with different systematic errors in the DH parameters and computed the mean error in translation and rotation over the 20 results. Finally, we repeated the whole experiment for different noise levels by multiplying the above standard deviations of the DH parameters by a noise factor ranging from 0 to 10, whereas keeping the noise in the image points constant. The results are

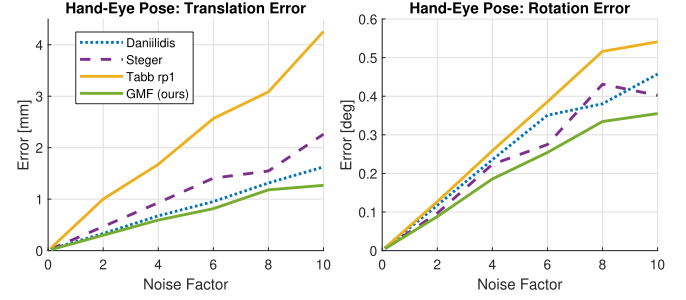


Fig. 7. Translation and rotation error of the resulting hand-eye pose for a robot with systematic errors in the DH parameters.

TABLE I
RESULTS OF THE SIMULTANEOUS CAMERA CALIBRATION

Camera Parameter	Initial Values	Ground Truth Values	Estimated Values	Standard Deviation
f (mm)	8.0	8.4300	8.4303	0.0002
κ (m^{-2})	0.0	1000.00	999.92	0.54
s_x ($\mu\text{m Pixel}^{-1}$)	5.2	5.21000	5.20997	0.00003
s_y ($\mu\text{m Pixel}^{-1}$)*	5.2	5.20000	5.20000	0.00000
c_x (Pixel)	640.0	660.00	659.99	0.02
c_y (Pixel)	512.0	482.00	481.96	0.02

Parameters indicated by * are excluded from the calibration.

shown in Fig. 7. Note that no additional noise was added to the robot poses. The experiment shows that our approach is also able to cope with systematic robot errors very well.

For a noise factor of 1.0, we also tested the simultaneous estimation of the interior camera parameters. Table I shows the initial values for the camera parameters, the ground truth values, the results of our approach, and the standard deviations, which are also returned by our approach. It can be seen that the estimated camera parameters are very close to their ground true values. Also, the standard deviations that are returned by our approach are on the same order of magnitude as the errors in the parameters, and hence, are plausible.

B. Experiments on Real Data

For our experiments, we attached a 2448×2048 IDS U3-3280SE camera with an 8 mm lens at the end-effector of a 6-axis UR3e robot arm (0.5 m working radius). To test the target-based calibration, we put a planar calibration target with known control points⁶ in the workspace of the robot and acquired images at 25 different robot poses (Fig. 8, top). To test the self-calibration capabilities of our model, we replaced the calibration target by a planar print and acquired 25 images by using the same robot poses (Fig. 8, bottom).

Table II displays the reprojection root-mean squared error (RRMSE) and the reconstruction accuracy error (RAE), which is the average squared Euclidean error of the reconstructed calibration points, as proposed in [42]. For the evaluation, we

⁶Image coordinates of the circular control points are extracted by fitting ellipses to their subpixel-accurate image edges. This causes a bias in the point positions [52]. For most applications, this distortion is so small that it can be neglected. However, the bias can be removed with the approach described in [53] if necessary.

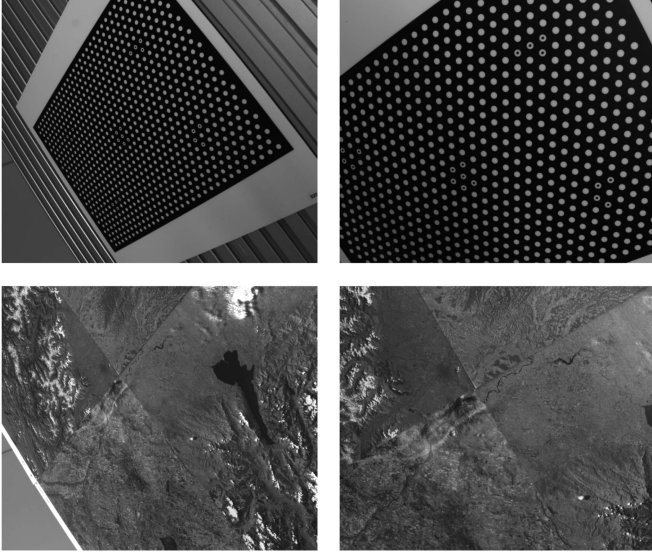


Fig. 8. Two example calibration images for the target-based hand-eye calibration (top row) and for hand-eye self-calibration (bottom row).

TABLE II
RRMSE AND RAE ERRORS ON THE UR3E DATA

Method	RRMSE (pixel)	RAE (mm ²)	Time (s)
Daniilidis [21]	0.708	0.00462	0.2
Steger [5]	0.608	0.00212	0.2
Strobl 1 [14]	1.082	0.04248	1.6
Strobl 2 [14]	1.152	0.07076	1.6
Tabb rp1 [42], fixed camera	0.567	0.00194	0.3
Tabb rp1 [42]	0.545	0.00212	0.3
GMF, orig. poses, fixed camera	0.784	0.02190	3.4
GMF, orig. poses	0.771	0.02103	3.4
GMF, fixed camera	0.196	0.00073	3.4
GMF	0.193	0.00073	3.4
GMF, self-calibration	0.178	-	14.2

compare different variants of GMF with the methods of Daniilidis [21], Steger [5], Tabb (rp1) [42], and Strobl [14] as a representative that models errors in the robot poses. For the evaluation of [14], we used the implementation of [54], which allows to choose whether the errors in the robot poses (Strobl 1) or in the camera poses (Strobl 2) are minimized. For GMF, “fixed camera” means that camera parameters are precalibrated separately by using [5, Ch. 3.9.4] and are excluded from the hand-eye calibration. “orig. poses” means that the errors are calculated with the original robot poses instead of the calibrated ones. “Self-calibration” means that no calibration target was used (in this case, RAE cannot be computed because the true 3-D points are unknown). For the GMF, we again initialized $\mathbf{Q}_U$ with $\sigma_i = 0.1$ pixels, $\sigma_a = 0.1^\circ$ and $\sigma_t = 1$ mm. For the calibrated robot poses, GMF returns the smallest errors for target-based calibration and for self-calibration.

Table II also shows the computation times of the different methods measured on an Intel Core i7-8565 U with 1.8 GHz. Compared with the methods of [21], [5], and [42], GMF is slower by a factor of approximately 10. This is mainly caused by the additional iteration that is needed in GMF for estimating the

TABLE III
RESULTING HAND-EYE POSES

Method	t_x (mm)	t_y (mm)	t_z (mm)	α ($^\circ$)	β ($^\circ$)	γ ($^\circ$)
GMF	63.64	135.21	45.67	2.26	25.39	176.35
GMF, self-calib	63.79	135.18	45.76	2.24	25.35	176.35

TABLE IV
ESTIMATED CAMERA PARAMETERS

Camera Parameter	Initial Values	Pre- Calibration	Simultaneous Calibration	Standard Deviation
f (mm)	8.0	8.143883	8.143442	0.000296
κ (m ⁻²)	0.0	-1828.362	-1834.429	2.176
s_x ($\mu\text{m pixel}^{-1}$)	3.45	3.449968	3.449982	0.000026
s_y ($\mu\text{m pixel}^{-1}$)*	3.45	3.450000	3.450000	0.000000
c_x (pixel)	1224.0	1249.083	1249.081	0.066
c_y (pixel)	1024.0	1035.332	1035.266	0.074

Parameters indicated by * are excluded from the calibration.

variance components. Nevertheless, the absolute value of 3.4 s is still fast enough for practical applications where the acquisition time of the calibration images takes by far the most time. In the self-calibration case, the computation time of GMF increases to 14.2 s because of the higher number of unknowns, i.e., the reconstructed 3-D points.

It is also interesting to have a look at the standard deviations of the image points, the robot translations, and the robot rotations, which are returned by GMF. They are 0.23 pixels, 0.093 mm, and 0.029° in the target-based calibration, and 0.32 pixels, 0.097 mm, and 0.025° in the self-calibration case. Thus, the robot uncertainty was estimated consistently. For comparison, the manufacturer specifies the repeatability of the robot as 0.03 mm. Consequently, our result suggests that the uncertainty of the robot is worse than its repeatability by a factor of 3, which is plausible. Furthermore, the image points of the calibration target are extracted with a slightly higher accuracy compared with the feature points in the prints, which is plausible as well. The resulting hand-eye poses for target-based calibration and self-calibration are shown in Table III.

We also compared the result of camera precalibration using the approach of [5, Ch. 3.9.4] with simultaneous camera calibration. Table IV shows the obtained camera parameters for both variants together with the standard deviations that are returned by our approach. In addition, the initial parameter values that were used for both variants are shown. Our results are consistent with the results obtained by precalibration.

To test the robustness of the estimation of the variance components to the initialization of σ_i , σ_a , and σ_t , we modified the standard values of 0.1 pixels, 0.1° , and 1 mm. Since only the ratio of the three values affects initialization, we independently multiplied σ_a and σ_t by a factor of 10^P where P is an integer ranging from -5 to +5, resulting in 11^2 combinations. For each combination, we used σ_i and the scaled versions of σ_a and σ_t to initialize $\mathbf{Q}_U$ and restarted our hand-eye calibration. For all experiments for which $-4 \leq P \leq 4$, the variance components successfully converged to the same values in at most five iterations. For $|P| = 5$, in some cases the least-squares estimation

TABLE V
INFORMATION ABOUT THE DATASETS PUBLISHED IN [42]

Dataset	Image Size (pixel $\times$ pixel)	Approx. Focal Length (mm)	Robot	n_r	Camera Calibration Error (pixel)
1	640 $\times$ 480	8	Denso VS-6577GM-B	88	0.185
2	2456 $\times$ 2058	8	Denso VS-6577GM-B	28	0.199
3	2456 $\times$ 2058	6	Denso VM-60BIG	36	0.540
4	1600 $\times$ 1200	6	Denso VM-60BIG	20	0.447

Camera calibration error refers to the reprojection RMSE of camera calibration.

TABLE VI
RRMSE AND RAE ERRORS ON THE DATASET [42] (RESULTS OF [42] AND [43] ARE TAKEN FROM [43])

	Dataset 1		Dataset 2		Dataset 3		Dataset 4	
Method	RRMSE (pixel)	RAE (mm ²)	RRMSE (pixel)	RAE (mm ²)	RRMSE (pixel)	RAE (mm ²)	RRMSE (pixel)	RAE (mm ²)
Daniilidis [21]	1.895	51.023	3.877	2.623	12.504	725.852	2.627	38.219
Steger [5]	1.411	5.096	3.578	0.070	3.624	9.356	1.524	2.082
Strobl 1 [14]	2.148	70.728	4.836	8.345	13.759	447.813	6.378	64.998
Strobl 2 [14]	2.888	111.251	4.945	9.971	13.748	515.157	6.986	129.097
Tabb c2 [42]	1.672	5.820	3.960	0.086	3.553	1.927	2.388	1.715
Tabb rp1 [42]	1.569	0.238	3.741	0.030	2.890	1.454	1.603	1.437
Tabb rp2 [42]	1.363	0.524	3.181	0.450	2.680	1.588	1.505	1.481
Koide Pose Graph [43]	1.562	0.214	3.751	0.038	2.879	1.337	1.619	1.380
GMF, orig. poses	1.567	7.715	3.835	3.519	11.744	584.722	1.661	9.090
GMF	0.076	0.113	0.176	0.031	0.479	1.188	0.389	1.380

failed due to numeric problems, which is not surprising since the values in $\mathbf{Q}_U$ differed by up to 11 orders of magnitude. The experiment shows that the estimation of the variance components is extremely robust, even for unrealistically incorrect initializations.

We also applied our approach to four datasets published in [42]. Information about these datasets is given in Table V and the results are shown in Table VI. For calibrated robot poses, GMF returns the smallest errors in almost all cases. When taking the original robot poses (GMF, orig. poses), the error is significantly higher.

C. Comprehensive Comparison With Previous Work

For a comprehensive comparison of our framework to previous methods, we use the publicly available benchmark dataset in [33]. We selected the CS_Synthetic_3 dataset for evaluation because it is the most comprehensive one and contains the greatest variation in the orientations of the robot and the camera, which is important for accurate hand-eye calibration [16]. It is simulated with Blender [55] and consists of 30 images of a checkerboard, which is captured from different poses, and the corresponding robot poses.

The authors in [33] aim to introduce realistic noise to get as close as possible to real data. They add Gaussian visual noise to the image points and introduce pseudorealistic noise to the robot poses, which consists of a systematic and a random (Gaussian) component and is different for each axis. The noise magnitude of the components is derived from a real robot by reverse engineering. The size of the systematic components is 0.06, -0.05 , and -0.04 mm for the robot position in X, Y, and Z as well as 0.0032° , -0.0002° , and 0.0002° for the robot rotation about these axes. The standard deviation of the random components is 0.22, 0.18, and 0.17 mm for the position in X, Y, and Z as well as 0.0177° , 0.0161° , and 0.0110° for the rotation about these axes.

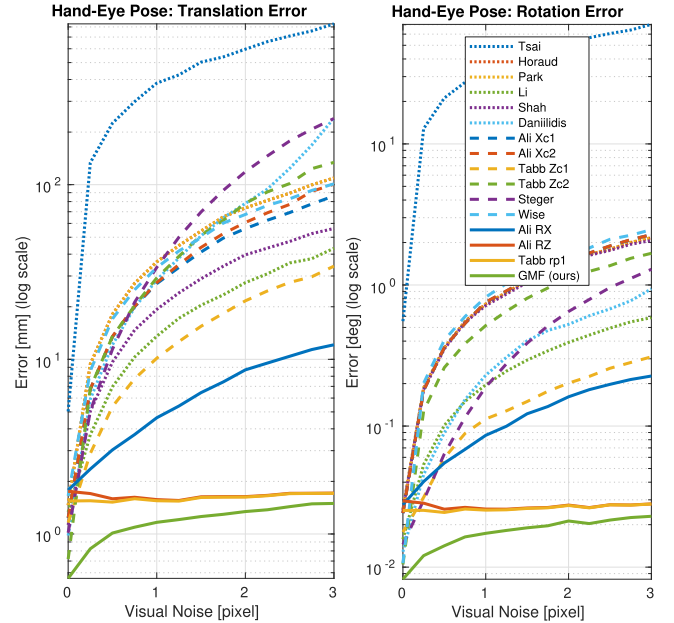


Fig. 9. Translation and rotation error of the hand-eye pose for varying visual noise and constant pseudorealistic robot pose noise calculated on the benchmark dataset of [33]. Dotted lines visualize linear approaches, dashed lines nonlinear approaches, and solid lines nonlinear approaches that minimize the reprojection error.

Note that our proposed framework models the robot noise by a single random component. However, it can be easily extended to more complex noise models in a straightforward way.

Fig. 9 visualizes the translation and rotation errors⁷ defined in [33] of the hand-eye pose for varying visual noise. For this experiment, the pseudorealistic robot pose noise was set

⁷Note that the definition of the rotation error of [33] slightly differs from our definition in Section IV-A.

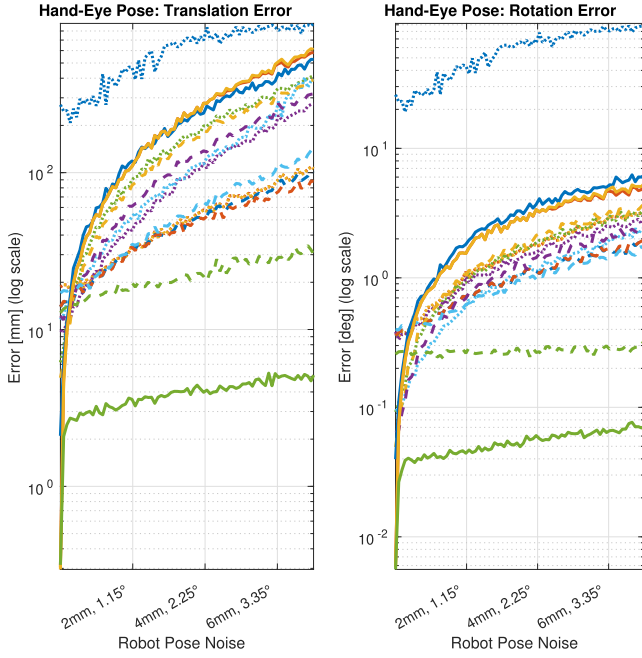


Fig. 10. Translation and rotation error of the hand-eye pose for varying robot pose noise and constant visual noise calculated on the benchmark dataset of [33] (see Fig. 9 for the legend).

to the above fixed values. For each level of visual noise, the computations were repeated 1000 times with different random noise. The mean results over all repetitions are shown in Fig. 9. Dotted lines visualize linear approaches: Tsai [16], Horaud [19], Park [28], Li [24], Shah [25], Daniilidis [21]. Dashed lines visualize nonlinear approaches: Ali Xc1 and Xc2 [33], Tabb Zc1 and Zc2 [42], Steger [5], Wise (R+C+H) [40]. Solid lines visualize nonlinear approaches that minimize the reprojection error: Ali RX and RZ [33], Tabb rp1 [42], GMF (ours) with a fixed camera. The implementations of the approaches are taken from the benchmark in [33] or from the source that is given in the respective publication.

Our framework clearly outperforms all previous approaches. The evaluation also shows that all approaches that minimize the reprojection error are more robust to visual noise, which agrees with the results of [33]. Of the linear methods, Li [24] is the most robust to high visual noise.

Fig. 10 visualizes the translation and rotation error defined in [33] of the hand-eye pose for varying robot pose noise. Now, the visual noise was set to a fixed value of 0.5 pixels. Rotation and translation components of the robot pose noise were varied simultaneously. For each level of robot pose noise, the computations were repeated 100 times with different random noise. The mean results over all repetitions are shown in Fig. 10. It shows that our framework outperforms all other approaches also in the presence of robot pose noise. In this experiment, the performance gain to the second best method is almost one order of magnitude. All other approaches that minimize the reprojection error are sensitive to robot pose noise. Here, approaches that minimize a pose error might have an advantage. This clearly shows the benefit of explicitly considering the robot uncertainty during

hand-eye calibration as our approach minimizes both visual and robot pose noise.

D. Discussion and Future Work

At this point, we would like to highlight again the advantages of estimating absolute values for the uncertainties. The result matrix $\mathbf{C}_U$ contains the uncertainty of the robot, which includes both absolute accuracy and repeatability. This information is usually not available since most robot manufacturers only specify the repeatability. For vision-guided robots, however, it is not the repeatability but the total uncertainty of the robot that is crucial. Therefore, our method can be used to easily determine the robot uncertainty, which in turn can be used, for example, to decide whether the robot is suited for a specific application with a certain accuracy requirement. The absolute accuracy of the hand-eye pose, which is returned in $\mathbf{C}_{\hat{x}\hat{x}}$, is of interest in practical applications of a vision-guided robot as well for the same reason. The accuracy information can be the base for the decision whether the accuracy of the estimated hand-eye pose is sufficiently high for a certain application. The absolute accuracy of the hand-eye pose might also be helpful to analyze whether the robot poses cover enough variations for an accurate hand-eye calibration. If, for example, all robot tool poses were more or less parallel to each other, the standard deviation in t_z would be very high. Finally, the obtained uncertainty information about the robot and the hand-eye pose is valuable as input for applying error propagation. For example, the robot can be used for stereo reconstruction by moving it to two specified different poses and acquiring an image at each pose. The uncertainty of the robot pose and that of the hand-eye pose can then be propagated together with the uncertainty of the measured image points to the uncertainty of the reconstructed 3-D point.

We model the uncertainty of the robot by assuming that the errors in the robot poses are zero-mean Gaussian distributed. While tests on real systems showed the validity of this assumption [14], the assumption might be violated for some robots. The evaluations presented in this section have shown that the accuracy of the hand-eye calibration benefits from modeling the uncertainty in the proposed way even when the distribution of the errors in the robot poses deviates from a zero-mean Gaussian distribution. Obviously, a wrong assumption about the exact shape of the error distribution is still better than ignoring the errors completely. However, the modeling and the accuracy of the hand-eye calibration could be further improved by directly modeling the uncertainty of the robot parameters of the individual joints rather than the robot pose, which result from the robot parameters by applying the forward kinematics. This is a topic for future research.

Our current implementation assumes that there are no outliers in the observations. For target-based calibration, the robust calibration mark extraction ensures that outliers are suppressed. For self-calibration, outliers are already identified and eliminated during the SfM pipeline, e.g., by using RANSAC [56]. Consequently, our optimization framework does not need to deal with outliers in both cases. However, if the correspondences would be obtained in a different way where outliers cannot

Generic GMF Algorithm for Hand-Eye Calibration

```

1: procedure GMF(Bool DoTargetBasedCalibration, Bool DoPrecalibrateCamera)
2:   if DoPrecalibrateCamera then
3:     Perform camera calibration  $\rightarrow$  interior orientation  $i$ 
4:   end if
5:   if DoTargetBasedCalibration then
6:     Place calibration target with  $n_w$  3D points  $p_k$  ( $k = 1, \dots, n_w$ ) in robot workspace
7:     for  $j = 1$  to  $n_r$  do
8:       Move robot tool to predefined pose  ${}^t\mathbf{H}_{b,j}^{(0)}$ 
9:       Acquire camera image
10:      Extract coordinates of calibration points  $p_{j,k}$  in the acquired image
11:      Compute  ${}^c\mathbf{H}_{w,j}$  from correspondences  $p_k, p_{j,k}$ , and  $i$  with PnP algorithm
12:    end for
13:   else
14:     Provide textured scene elements in robot workspace
15:     for  $j = 1$  to  $n_r$  do
16:       Move robot tool to pose  ${}^t\mathbf{H}_{b,j}^{(0)}$ 
17:       Acquire and save camera image
18:     end for
19:     Apply SfM pipeline (e.g., COLMAP) to camera images  $\rightarrow i^{(0)}, n_w, p_k^{(0)}, p_{j,k}, {}^c\mathbf{H}_{w,j}$ 
20:     Fix the SfM scale ambiguity in  $p_k^{(0)}$  and  ${}^c\mathbf{H}_{w,j}$  as described in Section III-G
21:   end if
22:   Perform linear hand-eye calibration of [21] based on  ${}^t\mathbf{H}_{b,j}^{(0)}$  and  ${}^c\mathbf{H}_{w,j} \rightarrow$  initial values for  ${}^c\mathbf{H}_t^{(0)}, {}^b\mathbf{H}_w^{(0)}$ 
23:   Build parameter vectors  ${}^c\mathbf{H}_t^{(0)} \rightarrow e_c^{(0)}, {}^b\mathbf{H}_w^{(0)} \rightarrow e_b^{(0)}, {}^t\mathbf{H}_{b,j}^{(0)} \rightarrow e_{t,j}^{(0)}$   $\triangleright$  for self-calibration,  ${}^b\mathbf{H}_w^{(0)} = \mathbf{I}$ 
24:   Set vector of observations  $l = (p_{1,1}^\top, \dots, p_{1,n_w}^\top, \dots, p_{n_r,n_w}^\top, e_{t,1}^{(0)}, \dots, e_{t,n_r}^{(0)})^\top$ 
25:   Build vector of unknowns  $x^{(0)} \leftarrow (e_c^{(0)\top}, e_{t,1}^{(0)\top}, \dots, e_{t,n_r}^{(0)\top})^\top$ 
26:   if not(DoPrecalibrateCamera) then
27:     Extend vector of unknowns  $x^{(0)} \leftarrow (x^{(0)}, i^{(0)\top})^\top$ 
28:   end if
29:   if DoTargetBasedCalibration then
30:     Extend vector of unknowns  $x^{(0)} \leftarrow (x^{(0)}, e_b^{(0)\top})^\top$ 
31:   else
32:     Extend vector of unknowns  $x^{(0)} \leftarrow (x^{(0)}, p_1^{(0)\top}, \dots, p_{n_w}^{(0)\top})^\top$ 
33:   end if
34:   Initialize variances of the three observation groups  $\sigma_i^2 \leftarrow 0.1^2$  pixel,  $\sigma_a^2 \leftarrow 0.1^2$  deg,  $\sigma_t^2 \leftarrow 1.0^2$  mm
35:   repeat
36:     Build  $\mathbf{Q}_l$  by using Eq. (16) and compute  $\mathbf{P}_l \leftarrow \mathbf{Q}_l^{-1}$ 
37:     Initialize vector of unknowns  $\hat{x} \leftarrow x^{(0)}$ 
38:     repeat
39:        $\Delta l \leftarrow l - f(\hat{x})$   $\triangleright f(\hat{x})$  is Eq. (10) extended by the current values of the known robot poses
40:       Compute Jacobian  $\mathbf{A}$  shown in Fig. 3 by using Eq. (11) and the partial derivatives given in the Appendix
41:       Solve Eq. (12) for  $\Delta \hat{x}$ 
42:        $\hat{x} \leftarrow \hat{x} + \Delta \hat{x}$ 
43:     until convergence of the parameter estimation ( $\Delta \hat{x} == 0$ )
44:     Compute  $\mathbf{Q}_{\hat{x}\hat{x}}, \mathbf{Q}_{\hat{x}l}, \mathbf{Q}_{ll}$ , and  $\mathbf{R}$  with Eqs. (19), (20), (21), and (22), respectively
45:     Compute the variance components  $\hat{\sigma}_{0,i}^2, \hat{\sigma}_{0,a}^2$ , and  $\hat{\sigma}_{0,t}^2$  with Eqs. (23) and update  $\sigma_i^2, \sigma_a^2, \sigma_t^2$  with Eqs. (24)
46:   until convergence of the variance component estimation ( $\hat{\sigma}_{0,i}^2 == \hat{\sigma}_{0,a}^2 == \hat{\sigma}_{0,t}^2 == 1 \Rightarrow \hat{\sigma}_0^2 = 1$ )
47:   Compute the covariances  $\mathbf{C}_l$  of the original observations with Eq. (13)  $\triangleright$  provides information about robot uncertainty
48:   Compute the covariances  $\mathbf{C}_{\hat{x}\hat{x}}$  of the estimated unknowns with Eq. (15)
49:   return  $\hat{x}, \mathbf{C}_{\hat{x}\hat{x}}, \mathbf{C}_l$ 
50: end procedure

```

Fig. 11. Pseudocode of our generic uncertainty-aware hand-eye calibration (GMF) for the cases target-based hand-eye calibration (DoTargetBasedCalibration == True), self-calibration (DoTargetBasedCalibration == False), using a precalibrated camera (DoPrecalibrateCamera == True), and simultaneous camera calibration (DoPrecalibrateCamera == False).

be suppressed completely, it would be necessary to extend the optimization by well-established methods for robust estimation. A comprehensive overview about robust estimation and outlier detection is given in [46, Ch. 4.7].

We currently assume that all image points are equally accurate. While this is a common assumption in computer vision, the assumption does not perfectly hold in practice. Therefore, we

would like to investigate whether considering a point-specific accuracy improves the accuracy of the hand-eye calibration. This could be easily established in our framework by setting individual entries in $\mathbf{Q}_l$ including covariances of row and column coordinates. For target-based calibration, we could use the size and shape of the elliptic projections of the circular calibration marks in the image to derive an individual variance

TABLE VII
DERIVATIVES OF STEP 1: TRANSFORMATION FROM WCS TO BCS

	$\partial x_b / \dots$	$\partial y_b / \dots$	$\partial z_b / \dots$
$\dots / \partial^b \alpha_w$	0	$-x_w(\sin^b \alpha_w \sin^b \gamma_w - \cos^b \alpha_w \cos^b \gamma_w \sin^b \beta_w)$ $-y_w(\cos^b \gamma_w \sin^b \alpha_w + \cos^b \alpha_w \sin^b \beta_w \sin^b \gamma_w)$ $-z_b \cos^b \alpha_w \cos^b \beta_w$	$x_w(\cos^b \alpha_w \sin^b \gamma_w + \cos^b \gamma_w \sin^b \alpha_w \sin^b \beta_w)$ $+y_w(\cos^b \alpha_w \cos^b \gamma_w - \sin^b \alpha_w \sin^b \beta_w \sin^b \gamma_w)$ $-z_b \cos^b \beta_w \sin^b \alpha_w$
$\dots / \partial^b \beta_w$	$-x_w \cos^b \gamma_w \sin^b \beta_w$ $+y_w \sin^b \beta_w \sin^b \gamma_w$ $+z_w \cos^b \beta_w$	$x_w \cos^b \beta_w \cos^b \gamma_w \sin^b \alpha_w$ $-y_w \cos^b \beta_w \sin^b \alpha_w \sin^b \gamma_w$ $+z_w \sin^b \alpha_w \sin^b \beta_w$	$-x_w \cos^b \alpha_w \cos^b \beta_w \cos^b \gamma_w$ $+y_w \cos^b \alpha_w \cos^b \beta_w \sin^b \gamma_w$ $-z_w \cos^b \alpha_w \sin^b \beta_w$
$\dots / \partial^b \gamma_w$	$-x_w \cos^b \beta_w \sin^b \gamma_w$ $-y_w \cos^b \beta_w \cos^b \gamma_w$	$x_w(\cos^b \alpha_w \cos^b \gamma_w - \sin^b \alpha_w \sin^b \beta_w \sin^b \gamma_w)$ $-y_w(\cos^b \alpha_w \sin^b \gamma_w + \cos^b \gamma_w \sin^b \alpha_w \sin^b \beta_w)$	$x_w(\cos^b \gamma_w \sin^b \alpha_w + \cos^b \alpha_w \sin^b \beta_w \sin^b \gamma_w)$ $-y_w(\sin^b \alpha_w \sin^b \gamma_w - \cos^b \alpha_w \cos^b \gamma_w \sin^b \beta_w)$
$\dots / \partial^b t x_w$	1	0	0
$\dots / \partial^b t y_w$	0	1	0
$\dots / \partial^b t z_w$	0	0	1
$\dots / \partial x_w$	$\cos^b \beta_w \cos^b \gamma_w$	$\cos^b \alpha_w \sin^b \gamma_w + \cos^b \gamma_w \sin^b \alpha_w \sin^b \beta_w$	$\sin^b \alpha_w \sin^b \gamma_w - \cos^b \alpha_w \cos^b \gamma_w \sin^b \beta_w$
$\dots / \partial y_w$	$-\cos^b \beta_w \sin^b \gamma_w$	$\cos^b \alpha_w \cos^b \gamma_w - \sin^b \alpha_w \sin^b \beta_w \sin^b \gamma_w$	$\cos^b \gamma_w \sin^b \alpha_w + \cos^b \alpha_w \sin^b \beta_w \sin^b \gamma_w$
$\dots / \partial z_w$	$\sin^b \beta_w$	$-\cos^b \beta_w \sin^b \alpha_w$	$\cos^b \alpha_w \cos^b \beta_w$

TABLE VIII
DERIVATIVES OF STEP 2: TRANSFORMATION FROM BCS TO TCS

	$\partial x_t / \dots$	$\partial y_t / \dots$	$\partial z_t / \dots$
$\dots / \partial^b \alpha_w$	$\frac{\partial x_t}{\partial y_b} \frac{\partial y_b}{\partial^b \alpha_w} + \frac{\partial x_t}{\partial z_b} \frac{\partial z_b}{\partial^b \alpha_w}$	$\frac{\partial y_t}{\partial y_b} \frac{\partial y_b}{\partial^b \alpha_w} + \frac{\partial y_t}{\partial z_b} \frac{\partial z_b}{\partial^b \alpha_w}$	$\frac{\partial z_t}{\partial y_b} \frac{\partial y_b}{\partial^b \alpha_w} + \frac{\partial z_t}{\partial z_b} \frac{\partial z_b}{\partial^b \alpha_w}$
$\dots / \partial^b \beta_w$	$\frac{\partial x_t}{\partial x_b} \frac{\partial x_b}{\partial^b \beta_w} + \frac{\partial x_t}{\partial y_b} \frac{\partial y_b}{\partial^b \beta_w} + \frac{\partial x_t}{\partial z_b} \frac{\partial z_b}{\partial^b \beta_w}$	$\frac{\partial y_t}{\partial x_b} \frac{\partial x_b}{\partial^b \beta_w} + \frac{\partial y_t}{\partial y_b} \frac{\partial y_b}{\partial^b \beta_w} + \frac{\partial y_t}{\partial z_b} \frac{\partial z_b}{\partial^b \beta_w}$	$\frac{\partial z_t}{\partial x_b} \frac{\partial x_b}{\partial^b \beta_w} + \frac{\partial z_t}{\partial y_b} \frac{\partial y_b}{\partial^b \beta_w} + \frac{\partial z_t}{\partial z_b} \frac{\partial z_b}{\partial^b \beta_w}$
$\dots / \partial^b \gamma_w$	$\frac{\partial x_t}{\partial x_b} \frac{\partial x_b}{\partial^b \gamma_w} + \frac{\partial x_t}{\partial y_b} \frac{\partial y_b}{\partial^b \gamma_w} + \frac{\partial x_t}{\partial z_b} \frac{\partial z_b}{\partial^b \gamma_w}$	$\frac{\partial y_t}{\partial x_b} \frac{\partial x_b}{\partial^b \gamma_w} + \frac{\partial y_t}{\partial y_b} \frac{\partial y_b}{\partial^b \gamma_w} + \frac{\partial y_t}{\partial z_b} \frac{\partial z_b}{\partial^b \gamma_w}$	$\frac{\partial z_t}{\partial x_b} \frac{\partial x_b}{\partial^b \gamma_w} + \frac{\partial z_t}{\partial y_b} \frac{\partial y_b}{\partial^b \gamma_w} + \frac{\partial z_t}{\partial z_b} \frac{\partial z_b}{\partial^b \gamma_w}$
$\dots / \partial^b t x_w$	$\frac{\partial x_t}{\partial x_b}$	$\frac{\partial y_t}{\partial x_b}$	$\frac{\partial z_t}{\partial x_b}$
$\dots / \partial^b t y_w$	$\frac{\partial x_t}{\partial y_b}$	$\frac{\partial y_t}{\partial y_b}$	$\frac{\partial z_t}{\partial y_b}$
$\dots / \partial^b t z_w$	$\frac{\partial x_t}{\partial z_b}$	$\frac{\partial y_t}{\partial z_b}$	$\frac{\partial z_t}{\partial z_b}$
$\dots / \partial x_w$	$\frac{\partial x_t}{\partial x_b} \frac{\partial x_b}{\partial x_w} + \frac{\partial x_t}{\partial y_b} \frac{\partial y_b}{\partial x_w} + \frac{\partial x_t}{\partial z_b} \frac{\partial z_b}{\partial x_w}$	$\frac{\partial y_t}{\partial x_b} \frac{\partial x_b}{\partial x_w} + \frac{\partial y_t}{\partial y_b} \frac{\partial y_b}{\partial x_w} + \frac{\partial y_t}{\partial z_b} \frac{\partial z_b}{\partial x_w}$	$\frac{\partial z_t}{\partial x_b} \frac{\partial x_b}{\partial x_w} + \frac{\partial z_t}{\partial y_b} \frac{\partial y_b}{\partial x_w} + \frac{\partial z_t}{\partial z_b} \frac{\partial z_b}{\partial x_w}$
$\dots / \partial y_w$	$\frac{\partial x_t}{\partial x_b} \frac{\partial x_b}{\partial y_w} + \frac{\partial x_t}{\partial y_b} \frac{\partial y_b}{\partial y_w} + \frac{\partial x_t}{\partial z_b} \frac{\partial z_b}{\partial y_w}$	$\frac{\partial y_t}{\partial x_b} \frac{\partial x_b}{\partial y_w} + \frac{\partial y_t}{\partial y_b} \frac{\partial y_b}{\partial y_w} + \frac{\partial y_t}{\partial z_b} \frac{\partial z_b}{\partial y_w}$	$\frac{\partial z_t}{\partial x_b} \frac{\partial x_b}{\partial y_w} + \frac{\partial z_t}{\partial y_b} \frac{\partial y_b}{\partial y_w} + \frac{\partial z_t}{\partial z_b} \frac{\partial z_b}{\partial y_w}$
$\dots / \partial z_w$	$\frac{\partial x_t}{\partial x_b} \frac{\partial x_b}{\partial z_w} + \frac{\partial x_t}{\partial y_b} \frac{\partial y_b}{\partial z_w} + \frac{\partial x_t}{\partial z_b} \frac{\partial z_b}{\partial z_w}$	$\frac{\partial y_t}{\partial x_b} \frac{\partial x_b}{\partial z_w} + \frac{\partial y_t}{\partial y_b} \frac{\partial y_b}{\partial z_w} + \frac{\partial y_t}{\partial z_b} \frac{\partial z_b}{\partial z_w}$	$\frac{\partial z_t}{\partial x_b} \frac{\partial x_b}{\partial z_w} + \frac{\partial z_t}{\partial y_b} \frac{\partial y_b}{\partial z_w} + \frac{\partial z_t}{\partial z_b} \frac{\partial z_b}{\partial z_w}$

scaling for each image point, for example. For self-calibration, an appropriate point extractor (e.g., [57]) could be used that returns information about the point's accuracy.

Because there is no bi-invariant distance metric in SE(3), i.e., the Euclidean group of rigid-body motions [58], there is a direction-dependence in our formulation of modeling the robot poses. Therefore, in future research we want to investigate whether this direction-dependence has a significant impact on the results, and, if so, investigate possible solutions.

V. CONCLUSION

Explicitly modeling the uncertainty of a robot is beneficial for robot hand-eye calibration. It improves the accuracy, returns calibrated robot poses, and gives information about the uncertainty of the robot. The proposed parameter estimation in the Gauss-Markov model with fictitious observations combined with the estimation of variance components enables a stochastically sound representation of the problem. Different scenarios of hand-eye calibration (e.g., target-based calibration, self-calibration, calibration of antropomorphic or SCARA robots, unknown or known interior orientation) can be easily represented by adding or removing the respective parameters to or from the parameter vectors. By this, a large number of applications can benefit from our approach. Implementation

details including several optimizations facilitate an easy, fast, and memory efficient realization of the framework making it applicable also in scenarios with a large number of calibration images and points. Finally, the experiments clearly demonstrate the advantages of our uncertainty-aware hand-eye calibration.

APPENDIX A PSEUDOCODE

Fig. 11 shows pseudocode of our generic uncertainty-aware hand-eye calibration, which uses parameter estimation in the Gauss-Markov model with fictitious observations (GMF).

APPENDIX B CALCULATION OF THE PARTIAL DERIVATIVES

A. Definitions

To write down the partial derivatives and to keep the notation as simple as possible, we use the following definitions:

- 1) $\mathbf{p}_w = (x_w, y_w, z_w)^\top$, $\mathbf{p}_b = (x_b, y_b, z_b)^\top$, $\mathbf{p}_t = (x_t, y_t, z_t)^\top$, and $\mathbf{p}_c = (x_c, y_c, z_c)^\top$ is a 3-D point in the WCS, BCS, TCS, and CCS, respectively.
- 2) $\mathbf{p}_i = (x_i, y_i)^\top$ is the corresponding projected 2-D image point.

TABLE IX
DERIVATIVES OF STEP 3: TRANSFORMATION FROM TCS TO CCS

[illegible]

TABLE X
DERIVATIVES OF STEP 4: PROJECTION INTO THE IMAGE PLANE (I)

	$\partial x_u/\dots$	$\partial y_u/\dots$
$\dots/\partial c$	$\frac{x_c}{z_c}$	$\frac{y_c}{z_c}$
$\dots/\partial x_c$	$\frac{c}{z_c}$	0
$\dots/\partial y_c$	0	$\frac{c}{z_c}$
$\dots/\partial z_c$	$-\frac{cx_c}{z_c^2}$	$-\frac{cy_c}{z_c^2}$

TABLE XI
DERIVATIVES OF STEP 4: PROJECTION INTO THE IMAGE PLANE (II)

[illegible]

TABLE XII
DERIVATIVES OF STEP 5: APPLYING LENS DISTORTIONS (I)

	$\partial x_d/\dots$	$\partial y_d/\dots$
$\dots/\partial\kappa$	$\frac{4x_u r_u^2}{D}$	$\frac{4y_u r_u^2}{D}$
$\dots/\partial x_u$	$\frac{2}{1+\sqrt{1-4\kappa r_u^2}} + \frac{8\kappa x_u^2}{D}$	$\frac{8\kappa x_u y_u}{D}$
$\dots/\partial y_u$	$\frac{8\kappa x_u y_u}{D}$	$\frac{2}{1+\sqrt{1-4\kappa r_u^2}} + \frac{8\kappa y_u^2}{D}$

- 3) The 3-D rigid transformation of a point in the WCS to the BCS is given by $\mathbf{p}_b = {}^b\mathbf{R}_w \mathbf{p}_w + {}^b\mathbf{t}_w$ with the translation vector ${}^b\mathbf{t}_w = ({}^b t_{wx}, {}^b t_{wy}, {}^b t_{wz})^\top$ and the rotation matrix ${}^b\mathbf{R}_w = \mathbf{R}_x({}^b\alpha_w) \mathbf{R}_y({}^b\beta_w) \mathbf{R}_z({}^b\gamma_w)$. The rotation matrices are parameterized by Euler angles

$$\mathbf{R}_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{pmatrix} \quad (25)$$

$$\mathbf{R}_y(\beta) = \begin{pmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{pmatrix} \quad (26)$$

$$\mathbf{R}_z(\gamma) = \begin{pmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{pmatrix}. \quad (27)$$

The 3-D rigid transformations $\mathbf{p}_t = {}^t\mathbf{R}_b\mathbf{p}_b + {}^t\mathbf{t}_b$ and $\mathbf{p}_c = {}^c\mathbf{R}_t\mathbf{p}_t + {}^c\mathbf{t}_t$ are defined accordingly.

B. Observation Equations

To build the Jacobian $\mathbf{A}$, we essentially need the partial derivatives of the observation equations (10) with respect to the unknowns. With the previously introduced notation, equation (10) can be split up into the following functions that are executed consecutively:

TABLE XIII
DERIVATIVES OF STEP 5: APPLYING LENS DISTORTIONS (II)

	$\partial x_d / \dots$	$\partial y_d / \dots$
$\dots / \partial^b \alpha_w$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^b \alpha_w} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^b \alpha_w}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^b \alpha_w} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^b \alpha_w}$
$\dots / \partial^b \beta_w$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^b \beta_w} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^b \beta_w}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^b \beta_w} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^b \beta_w}$
$\dots / \partial^b \gamma_w$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^b \gamma_w} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^b \gamma_w}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^b \gamma_w} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^b \gamma_w}$
$\dots / \partial^b t_{xw}$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^b t_{xw}} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^b t_{xw}}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^b t_{xw}} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^b t_{xw}}$
$\dots / \partial^b t_{yw}$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^b t_{yw}} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^b t_{yw}}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^b t_{yw}} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^b t_{yw}}$
$\dots / \partial^b t_{zw}$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^b t_{zw}} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^b t_{zw}}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^b t_{zw}} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^b t_{zw}}$
$\dots / \partial^t \alpha_b$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^t \alpha_b} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^t \alpha_b}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^t \alpha_b} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^t \alpha_b}$
$\dots / \partial^t \beta_b$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^t \beta_b} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^t \beta_b}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^t \beta_b} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^t \beta_b}$
$\dots / \partial^t \gamma_b$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^t \gamma_b} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^t \gamma_b}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^t \gamma_b} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^t \gamma_b}$
$\dots / \partial^t t_{xb}$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^t t_{xb}} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^t t_{xb}}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^t t_{xb}} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^t t_{xb}}$
$\dots / \partial^t t_{yb}$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^t t_{yb}} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^t t_{yb}}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^t t_{yb}} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^t t_{yb}}$
$\dots / \partial^t t_{zb}$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^t t_{zb}} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^t t_{zb}}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^t t_{zb}} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^t t_{zb}}$
$\dots / \partial^c \alpha_t$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^c \alpha_t} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^c \alpha_t}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^c \alpha_t} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^c \alpha_t}$
$\dots / \partial^c \beta_t$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^c \beta_t} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^c \beta_t}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^c \beta_t} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^c \beta_t}$
$\dots / \partial^c \gamma_t$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^c \gamma_t} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^c \gamma_t}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^c \gamma_t} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^c \gamma_t}$
$\dots / \partial^c t_{xt}$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^c t_{xt}} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^c t_{xt}}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^c t_{xt}} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^c t_{xt}}$
$\dots / \partial^c t_{yt}$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^c t_{yt}} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^c t_{yt}}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^c t_{yt}} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^c t_{yt}}$
$\dots / \partial^c t_{zt}$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial^c t_{zt}} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial^c t_{zt}}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial^c t_{zt}} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial^c t_{zt}}$
$\dots / \partial c$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial c} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial c}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial c} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial c}$
$\dots / \partial x_w$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial x_w} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial x_w}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial x_w} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial x_w}$
$\dots / \partial y_w$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial y_w} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial y_w}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial y_w} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial y_w}$
$\dots / \partial z_w$	$\frac{\partial x_d}{\partial x_u} \frac{\partial x_u}{\partial z_w} + \frac{\partial x_d}{\partial y_u} \frac{\partial y_u}{\partial z_w}$	$\frac{\partial y_d}{\partial x_u} \frac{\partial x_u}{\partial z_w} + \frac{\partial y_d}{\partial y_u} \frac{\partial y_u}{\partial z_w}$

TABLE XIV
DERIVATIVES OF STEP 6: TRANSFORMATION INTO THE IMAGE COORDINATE SYSTEM (I)

	$\partial x_i / \dots$	$\partial y_i / \dots$
$\dots / \partial s_x$	$-x_d / s_x^2$	0
$\dots / \partial s_y$	0	$-y_d / s_y^2$
$\dots / \partial c_x$	1	0
$\dots / \partial c_y$	0	1
$\dots / \partial x_d$	$1 / s_x$	0
$\dots / \partial y_d$	0	$1 / s_y$

1) Transformation from WCS to BCS

$$\mathbf{p}_b = {}^b\mathbf{R}_w \mathbf{p}_w + {}^b\mathbf{t}_w. \quad (28)$$

2) Transformation from BCS to TCS

$$\mathbf{p}_t = {}^t\mathbf{R}_b \mathbf{p}_b + {}^t\mathbf{t}_b. \quad (29)$$

3) Transformation from TCS to CCS

$$\mathbf{p}_c = {}^c\mathbf{R}_t \mathbf{p}_t + {}^c\mathbf{t}_t. \quad (30)$$

4) Projection into the image plane

$$\begin{pmatrix} x_u \\ y_u \end{pmatrix} = \frac{c}{z_c} \begin{pmatrix} x_c \\ y_c \end{pmatrix}. \quad (31)$$

5) Applying lens distortions [inverse of (7)]

$$\begin{pmatrix} x_d \\ y_d \end{pmatrix} = \frac{2}{1 + \sqrt{1 - 4\kappa(x_u^2 + y_u^2)}} \begin{pmatrix} x_u \\ y_u \end{pmatrix}. \quad (32)$$

TABLE XV
DERIVATIVES OF STEP 6: TRANSFORMATION INTO THE IMAGE COORDINATE SYSTEM (II)

	$\partial x_i / \dots$	$\partial y_i / \dots$
$\dots / \partial^b \alpha_w$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^b \alpha_w} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^b \alpha_w}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^b \alpha_w} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^b \alpha_w}$
$\dots / \partial^b \beta_w$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^b \beta_w} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^b \beta_w}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^b \beta_w} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^b \beta_w}$
$\dots / \partial^b \gamma_w$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^b \gamma_w} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^b \gamma_w}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^b \gamma_w} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^b \gamma_w}$
$\dots / \partial^b t_{xw}$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^b t_{xw}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^b t_{xw}}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^b t_{xw}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^b t_{xw}}$
$\dots / \partial^b t_{yw}$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^b t_{yw}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^b t_{yw}}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^b t_{yw}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^b t_{yw}}$
$\dots / \partial^b t_{zw}$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^b t_{zw}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^b t_{zw}}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^b t_{zw}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^b t_{zw}}$
$\dots / \partial^t \alpha_b$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^t \alpha_b} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^t \alpha_b}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^t \alpha_b} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^t \alpha_b}$
$\dots / \partial^t \beta_b$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^t \beta_b} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^t \beta_b}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^t \beta_b} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^t \beta_b}$
$\dots / \partial^t \gamma_b$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^t \gamma_b} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^t \gamma_b}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^t \gamma_b} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^t \gamma_b}$
$\dots / \partial^t t_{xb}$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^t t_{xb}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^t t_{xb}}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^t t_{xb}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^t t_{xb}}$
$\dots / \partial^t t_{yb}$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^t t_{yb}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^t t_{yb}}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^t t_{yb}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^t t_{yb}}$
$\dots / \partial^t t_{zb}$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^t t_{zb}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^t t_{zb}}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^t t_{zb}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^t t_{zb}}$
$\dots / \partial^c \alpha_t$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^c \alpha_t} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^c \alpha_t}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^c \alpha_t} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^c \alpha_t}$
$\dots / \partial^c \beta_t$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^c \beta_t} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^c \beta_t}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^c \beta_t} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^c \beta_t}$
$\dots / \partial^c \gamma_t$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^c \gamma_t} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^c \gamma_t}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^c \gamma_t} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^c \gamma_t}$
$\dots / \partial^c t_{xt}$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^c t_{xt}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^c t_{xt}}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^c t_{xt}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^c t_{xt}}$
$\dots / \partial^c t_{yt}$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^c t_{yt}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^c t_{yt}}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^c t_{yt}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^c t_{yt}}$
$\dots / \partial^c t_{zt}$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial^c t_{zt}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^c t_{zt}}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial^c t_{zt}} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial^c t_{zt}}$
$\dots / \partial c$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial c} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial c}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial c} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial c}$
$\dots / \partial \kappa$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial \kappa} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial \kappa}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial \kappa} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial \kappa}$
$\dots / \partial x_w$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial x_w} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial x_w}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial x_w} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial x_w}$
$\dots / \partial y_w$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial y_w} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial y_w}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial y_w} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial y_w}$
$\dots / \partial z_w$	$\frac{\partial x_i}{\partial x_d} \frac{\partial x_d}{\partial z_w} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial z_w}$	$\frac{\partial y_i}{\partial x_d} \frac{\partial x_d}{\partial z_w} + \frac{\partial y_i}{\partial y_d} \frac{\partial y_d}{\partial z_w}$

6) Transformation into the image coordinate system

$$\begin{pmatrix} x_i \\ y_i \end{pmatrix} = \begin{pmatrix} x_d / s_x + c_x \\ y_d / s_y + c_y \end{pmatrix}. \quad (33)$$

Note that in step 5, we focus on the division model since the polynomial model cannot be inverted analytically. If the polynomial model should be used, its partial derivatives can be computed numerically. Alternatively, the minimization of the reprojection error can be performed in the (distorted) image plane instead of in the image coordinate system (for details, see [5, Ch. 3.9.4.2]).

In the following representations of the derivatives, zeros and ones are explicitly considered for efficiency reasons.

C. Derivatives of Step 1: Transformation From WCS to BCS

The partial derivatives of x_b , y_b , and z_b with respect to ${}^b\alpha_w$, ${}^b\beta_w$, ${}^b\gamma_w$, ${}^b t_{xw}$, ${}^b t_{yw}$, ${}^b t_{zw}$, x_w , y_w , and z_w are given in Table VII. Note that rows 1–3 of Table VII are the transposed of $[-{}^b\mathbf{R}_w \mathbf{p}_w] \times \mathbf{M}$ with

$$\mathbf{M} = \begin{pmatrix} 1 & 0 & \sin {}^b\beta_w \\ 0 & \cos {}^b\alpha_w & -\sin {}^b\alpha_w \cos {}^b\beta_w \\ 0 & \sin {}^b\alpha_w & \cos {}^b\alpha_w \cos {}^b\beta_w \end{pmatrix} \quad (34)$$

and $[\mathbf{x}]_{\times}$ denoting the skew-symmetric 3×3 matrix of $\mathbf{x}$ for that $[\mathbf{x}]_{\times} \mathbf{y} = \mathbf{x} \times \mathbf{y}$ holds (see [59] for details). Rows 4–6 obviously are the identity matrix and rows 7–9 are the transposed of ${}^b\mathbf{R}_w$.

D. Derivatives of Step 2: Transformation From BCS to TCS

The partial derivatives of x_t , y_t , and z_t with respect to ${}^t\alpha_b$, ${}^t\beta_b$, ${}^t\gamma_b$, ttx_b , tty_b , ttz_b , x_b , y_b , and z_b are analogous to those of Step 1. Then, we apply the chain rule to obtain the partial derivatives of x_t , y_t , and z_t with respect to the parameters of the previous steps (see Table VIII). Note that rows 1–3 of Table VIII are the transposed of $-{}^t\mathbf{R}_b[{}^b\mathbf{R}_w\mathbf{p}_w]_{\times}\mathbf{M}$, rows 3–5 are the transposed of ${}^t\mathbf{R}_b$, and rows 7–9 are the transposed of ${}^t\mathbf{R}_b{}^b\mathbf{R}_w$.

E. Derivatives of Step 3: Transformation From TCS to CCS

The partial derivatives of x_c , y_c , and z_c with respect to ${}^c\alpha_t$, ${}^c\beta_t$, ${}^c\gamma_t$, ctx_t , cty_t , ctz_t , x_t , y_t , and z_t are analogous to those of Step 1. Then, we apply the chain rule to obtain the partial derivatives of x_c , y_c , and z_c with respect to the parameters of the previous steps (see Table IX).

F. Derivatives of Step 4: Projection Into the Image Plane

The partial derivatives of x_u and y_u with respect to c , x_c , y_c , and z_c are shown in Table X.

Then, we apply the chain rule to obtain the partial derivatives of x_u and y_u with respect to the parameters of the previous steps (see Table XI).

G. Derivatives of Step 5: Applying Lens Distortions

The partial derivatives of x_d and y_d with respect to κ , x_u , and y_u are shown in Table XII with $r_u = \sqrt{x_u^2 + y_u^2}$ and $D = \sqrt{1 - 4\kappa r_u^2(1 + \sqrt{1 - 4\kappa r_u^2})^2}$.

Then, we apply the chain rule to obtain the partial derivatives of x_d and y_d with respect to the parameters of the previous steps (see Table XIII).

H. Derivatives of Step 6: Transformation Into the Image Coordinate System

The partial derivatives of x_i and y_i with respect to s_x , s_y , c_x , c_y , x_d , and y_d are shown in Table XIV.

Then, we apply the chain rule to obtain the partial derivatives of x_i and y_i with respect to the parameters of the previous steps (see Table XV). Thus, all partial derivatives that are required to build the Jacobian $\mathbf{A}$ are known.

ACKNOWLEDGMENT

The authors would like to thank the MVTec Software GmbH, where the first ideas on this research topic were developed and discussed. Furthermore, the authors would also like to thank their colleague Bastian Erdn    for his careful review of Appendix B and his valuable suggestions for improvement.

REFERENCES

- [1] A. Hofhauser, C. Steger, and N. Navab, "Perspective planar shape matching," in *Image Processing: Machine Vision Applications II*, vol. 7251, K. S. Niel and D. Fofi, Eds. Bellingham, WA, USA: SPIE, 2009.
- [2] S. Hinterstoisser et al., "Model based training, detection and pose estimation of texture-less 3 D objects in heavily cluttered scenes," in *Computer Vision*, K. M. Lee, Y. Matsushita, J. M. Rehg, and Z. Hu, Eds. Berlin, Germany: Springer, 2013, pp. 548–562.
- [3] M. Ulrich, C. Wiedemann, and C. Steger, "Combining scale-space and similarity-based aspect graphs for fast 3D object recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 10, pp. 1902–1914, Oct. 2012.
- [4] Y. Labb  , J. Carpentier, M. Aubry, and J. Sivic, "CosyPose: Consistent multi-view multi-object 6D pose estimation," in *Proc. Eur. Conf. Comput. Vis.*, A. Vedaldi, H. Bischof, T. Brox, JM. Frahm Eds. 2020, pp. 574–591.
- [5] C. Steger, M. Ulrich, and C. Wiedemann, *Machine Vision Algorithms and Applications*, 2nd ed. Hoboken, NJ, USA: Wiley, 2018.
- [6] ISO 9283:1998, "Manipulating industrial robots – Performance criteria and related test methods," Standard, International Organization for Standardization, Geneva, CH, Apr. 1998.
- [7] P. Shiakolas, K. Conrad, and T. Yih, "On the accuracy, repeatability, and degree of influence of kinematics parameters for industrial robots," *Int. J. Model. Simul.*, vol. 22, no. 4, pp. 245–254, 2002.
- [8] M. Abderrahim, A. Khamis, S. Garrido, and L. Moreno, "Accuracy and calibration issues of industrial manipulators," in *Industrial Robotics: Programming, Simulation and Application*. London, U.K.: IntechOpen, 2006, pp. 131–146.
- [9] M. Morozov et al., "Assessing the accuracy of industrial robots through metrology for the enhancement of automated non-destructive testing," in *Proc. IEEE Int. Conf. Multisensor Fusion Integration Intell. Syst.*, 2016, pp. 335–340.
- [10] M. Placzek and  . Piszczek, "Testing of an industrial robot-s accuracy and repeatability in off and online environments," *Eksplotacja i Niezawodnos   – Maintenance Rel.*, vol. 20, no. 3, pp. 455–464, 2018.
- [11] J. Denavit and R. S. Hartenberg, "A kinematic notation for lower-pair mechanisms based on matrices," *J. Appl. Mechanics*, vol. 22, no. 2, pp. 215–221, Jun. 1955.
- [12] M. Ulrich and M. Hillemann, "Generic hand-eye calibration of uncertain robots," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2021, pp. 11 060–11 066.
- [13] MVTec Softw. GmbH, "HALCON/HDevelop Reference Manual," HALCON version 23.05, May 2023. [Online]. Available: <https://www.mvtec.com/products/halcon>
- [14] K. H. Strobl and G. Hirzinger, "Optimal hand-eye calibration," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2006, pp. 4647–4653.
- [15] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 22, no. 11, pp. 1330–1334, Nov. 2000.
- [16] R. Y. Tsai and R. K. Lenz, "A new technique for fully autonomous and efficient 3D robotics hand/eye calibration," *IEEE Trans. Robot. Autom.*, vol. 5, no. 3, pp. 345–358, Jun. 1989.
- [17] J. C. K. Chou and M. Kamel, "Finding the position and orientation of a sensor on a robot manipulator using quaternions," *Int. J. Robot. Res.*, vol. 10, no. 3, pp. 240–254, 1991.
- [18] H. H. Chen, "A screw motion approach to uniqueness analysis of head-eye geometry," in *Proc. Comput. Vis. Pattern Recognit.*, 1991, pp. 145–151.
- [19] R. Horaud and F. Dornaika, "Hand-eye calibration," *Int. J. Robot. Res.*, vol. 14, no. 3, pp. 195–210, 1995.
- [20] F. Dornaika and R. Horaud, "Simultaneous robot-world and hand-eye calibration," *IEEE Trans. Robot. Autom.*, vol. 14, no. 4, pp. 617–622, Aug. 1998.
- [21] K. Daniilidis, "Hand-eye calibration using dual quaternions," *Int. J. Robot. Res.*, vol. 18, no. 3, pp. 286–298, 1999.
- [22] N. Andreff, R. Horaud, and B. Espiau, "Robot hand-eye calibration using structure-from-motion," *Int. J. Robot. Res.*, vol. 20, no. 3, pp. 228–248, 2001.
- [23] J. Schmidt, F. Vogt, and H. Niemann, "Robust hand-eye calibration of an endoscopic surgery robot using dual quaternions," in *Pattern Recognition*, B. Michaelis and G. Krell, Eds. Berlin, Germany: Springer, 2003, pp. 548–556.
- [24] A. Li, L. Wang, and D. Wu, "Simultaneous robot-world and hand-eye calibration using dual-quaternions and Kronecker product," *Int. J. Phys. Sci.*, vol. 5, no. 10, pp. 1530–1536, 2010.
- [25] M. Shah, "Solving the robot-world/hand-eye calibration problem using the Kronecker product," *J. Mechanisms Robot.*, vol. 5, no. 3, 2013, Art. no. 031007.

- [26] M. Ulrich and C. Steger, "Hand-eye calibration of SCARA robots using dual quaternions," *Pattern Recognit. Image Anal.*, vol. 26, no. 1, pp. 231–239, 2016.
- [27] J. Schmidt and H. Niemann, "Data selection for hand-eye calibration: A vector quantization approach," *Int. J. Robot. Res.*, vol. 27, no. 9, pp. 1027–1053, 2008.
- [28] F. C. Park and B. J. Martin, "Robot sensor calibration: Solving $AX=XB$ on the Euclidean group," *IEEE Trans. Robot. Autom.*, vol. 10, no. 5, pp. 717–721, Oct. 1994.
- [29] S. Rémy, M. Dhome, J. M. Lavest, and N. Daucher, "Hand-eye calibration," in *Proc. Int. Conf. Intell. Robots Syst.*, 1997, pp. 1057–1065.
- [30] G.-Q. Wei, K. Arbter, and G. Hirzinger, "Active self-calibration of robotic eyes and hand-eye relationships with model identification," *IEEE Trans. Robot. Autom.*, vol. 14, no. 1, pp. 158–166, Feb. 1998.
- [31] J. Schmidt, F. Vogt, and H. Niemann, "Calibration-free hand-eye calibration: A structure-from-motion approach," in *Pattern Recognition*, W. G. Kropatsch, R. Sablatnig, and A. Hanbury, Eds. Berlin, Germany: Springer, 2005, pp. 67–74.
- [32] J. Heller, M. Havlena, and T. Pajdla, "Globally optimal hand-eye calibration using branch-and-bound," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 5, pp. 1027–1033, May 2016.
- [33] I. Ali, O. Suominen, A. Gotchev, and E. R. Morales, "Methods for simultaneous robot-world-hand-eye calibration: A comparative study," *Sensors*, vol. 19, no. 12, 2019, Art. no. 2837.
- [34] I. Eneuse, M. Foo, B. S. K. Ibrahim, H. Ahmed, F. Supmak, and O. S. Eyobu, "A comparative review of hand-eye calibration techniques for vision guided robots," *IEEE Access*, vol. 9, pp. 113 143–113 155, 2021.
- [35] H. Nguyen and Q. Pham, "On the covariance of X in $ax = xB$," *IEEE Trans. Robot.*, vol. 34, no. 6, pp. 1651–1658, Dec. 2018.
- [36] J. Brookshire and S. Teller, "Extrinsic calibration from per-sensor egomotion," in *Robotics: Science and Systems VIII*. Cambridge, MA, USA: MIT Press, 2013, pp. 504–512.
- [37] J. Wu, Y. Sun, M. Wang, and M. Liu, "Hand-eye calibration: 4-D procrustes analysis approach," *IEEE Trans. Instrum. Meas.*, vol. 69, no. 6, pp. 2966–2981, Jun. 2020.
- [38] J. Wu, M. Liu, Y. Huang, C. Jin, Y. Wu, and C. Yu, "SE(n) : An efficient solution to multiple pose estimation problems," *IEEE Trans. Cybern.*, vol. 52, no. 5, pp. 3829–3840, May 2022.
- [39] J. H. D. Henrion and T. Pajdla, "Hand-eye and robot-world calibration by global polynomial optimization," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2014, pp. 3157–3164.
- [40] E. Wise, M. Giamou, S. Khoubyarian, A. Grover, and J. Kelly, "Certifiably optimal monocular hand-eye calibration," in *Proc. IEEE Int. Conf. Multisensor Fusion Integration Intell. Syst.*, 2020, pp. 271–278.
- [41] A. Malti, "Hand-eye calibration with epipolar constraints: Application to endoscopy," *Robot. Auton. Syst.*, vol. 61, no. 2, pp. 161–169, 2013.
- [42] A. Tabb and K. M. A. Yousef, "Solving the robot-world hand-eye(s) calibration problem with iterative methods," *Mach. Vis. Appl.*, vol. 28, no. 5, pp. 569–590, Aug. 2017.
- [43] K. Koide and E. Menegatti, "General hand-eye calibration based on reprojection error minimization," *IEEE Robot. Autom. Lett.*, vol. 4, no. 2, pp. 1021–1028, Apr. 2019.
- [44] R. Lenz and D. Fritsch, "Accuracy of videometry with CCD sensors," *ISPRS J. Photogrammetry Remote Sens.*, vol. 45, no. 2, pp. 90–110, 1990.
- [45] D. C. Brown, "Close-range camera calibration," *Photogrammetric Eng.*, vol. 37, no. 8, pp. 855–866, Aug. 1971.
- [46] W. Förstner and B. P. Wrobel, *Photogrammetric Computer Vision: Statistics, Geometry, Orientation and Reconstruction*. Berlin, Germany: Springer, 2016.
- [47] H. Zhuang, "Hand/eye calibration for electronic assembly robots," *IEEE Trans. Robot. Autom.*, vol. 14, no. 4, pp. 612–616, Aug. 1998.
- [48] K.-R. Koch, *Parameter Estimation and Hypothesis Testing in Linear Models*. Berlin, Germany: Springer, 1999.
- [49] K.-R. Koch, *Introduction to Bayesian Statistics*, 2nd ed. Berlin, Germany: Springer, 2007.
- [50] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 4104–4113.
- [51] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*, 2nd ed. Hoboken, NJ, USA: Wiley, 2020.
- [52] J. Mallon and P. F. Whelan, "Which pattern? Biasing aspects of planar calibration patterns and detection methods," *Pattern Recognit. Lett.*, vol. 28, no. 9, pp. 921–930, 2007.
- [53] C. Steger, "A comprehensive and versatile camera model for cameras with tilt lenses," *Int. J. Comput. Vis.*, vol. 123, no. 2, pp. 121–159, 2017.
- [54] K. H. Strobl, W. Sepp, S. Fuchs, C. Paredes, M. Smisek, and K. Arbter, "DLR CalDe and DLR CalLab," *Inst. Robot. Mechatron*, German Aerospace Center (DLR), Oberpfaffenhofen, Germany, Accessed: Oct. 17, 2023. [Online]. Available: <http://www.robotic.dlr.de/callab>
- [55] Blender Online Community, "Blender - A 3D Modelling and Rendering Package," 2022. Accessed: Feb. 10, 2022. [Online]. Available: <http://www.blender.org>
- [56] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [57] W. Förstner and E. Gülch, "A fast operator for detection and precise location of distinct point, corners and centres of circular features," in *Proc. ISPRS Conf. Fast Process. Photogrammetric Data*, 1987, pp. 281–305.
- [58] F. C. Park, "Distance metrics on the rigid-body motions with applications to mechanism design," *J. Mech. Des.*, vol. 117, no. 1, pp. 48–54, Mar. 1995.
- [59] B. Erdnuss, "Proper synchronization of geospatial metadata in motion imagery and its evaluation," in *Geospatial Informatics X*, P. J. Doucette, J. D. Harguess, K. Palaniappan, and G. Seetharaman, Eds., vol. 11398. Bellingham, WA, USA: SPIE, 2020, pp. 85–105.



Markus Ulrich received the doctorate degree in geodesy from the Chair of Photogrammetry and Remote Sensing, Technical University of Munich (TUM), Munich, Germany, in 2003.

Subsequently, he was a software Engineer with the Research and Development Department, MVTec Software GmbH, where he became Head of the research team, in 2008. From 2005 to 2020, he was a Guest Lecturer with TUM, where he taught close-range photogrammetry. From 2013 to 2020, he was a Guest Lecturer with the Karlsruhe Institute of Technology (KIT), where he taught machine vision. In 2017, he completed his habilitation (*venia legendi*) and was appointed a Privatdozent (Lecturer) with the Department of Civil Engineering, Geo and Environmental Sciences, KIT. Since 2020, he has been a Professor of machine vision metrology with the Department of Civil Engineering, Geo and Environmental Sciences, the Institute of Photogrammetry and Remote Sensing, KIT, Karlsruhe, Germany. His research areas include machine vision, close-range photogrammetry, image processing, machine learning, and their applications in industry.



Markus Hillemann received the doctorate degree in geodesy and geoinformatics from Department of Civil Engineering, Geo and Environmental Sciences, Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany, in 2020.

From 2016 to 2020, he was a Research Associate and a Ph.D. Student with the Institute of Photogrammetry and Remote Sensing (IPF), KIT, as well as an external employee with the Fraunhofer Institute of Optics, System Technologies, and Image Exploitation, Ettlingen, Germany. Subsequently, he joined the Machine Vision Metrology group at IPF as a Postdoc. His research interests include methodology development and applications in machine vision, image and point cloud processing, machine learning, and photogrammetry.